생체인식시스템 시험·인증 안내서

CONTENTS

I chapter 생체인식시스템 성능 시험·인증 소개 5

1. 개요 ····· 5
2. 진흥원 안내 ····· 6

II chapter 생체인식시스템 성능 시험·인증 절차 7

1. 개요 ····· 7
2. 시험 절차 ····· 8

III chapter 평가분야 18

1. 표준적합성 시험 ····· 18
2. 지문인식 성능시험 ····· 26
3. 얼굴인식 성능시험 ····· 32
4. 정맥인식 성능시험 ····· 40
5. 홍채인식 성능시험 ····· 46

표제목

표 2-1. 생체인식시스템 표준적합성 및 성능 시험・인증 절차 ····· 7
표 2-2. 사전 성능시험에 사용되는 얼굴인식 DB 구성정보 ····· 10
표 2-3. 사전 성능시험에 사용되는 지문인식 DB 구성정보 ····· 11
표 2-4. 사전 성능시험에 사용되는 홍채인식 DB 구성정보 ····· 11
표 2-5. 사전 성능시험에 사용되는 손등정맥인식 DB 구성정보 ····· 12
표 2-6. 사전 성능시험에 사용되는 지정맥인식 DB 구성정보 ····· 12
표 2-7. 사전 성능시험에 사용되는 장정맥인식 DB 구성정보 ····· 13
표 2-8. 표준적합성 사전시험 결과표 ····· 14
표 2-9. 생체인식 성능 인증기준 ····· 16
표 2-10. 성능 시험항목 계산식 ····· 16
표 3-1. BSP 제공 기능에 따른 시험 시나리오 ····· 19
표 3-2. SPI 함수 기능 ····· 22
표 3-3. 지문인식 센서 종류 ····· 27
표 3-4. 지문인식 성능시험용 DB 구성정보 ····· 30
표 3-5. 얼굴인식 성능시험용 DB 구성정보 ····· 38
표 3-6. 손등정맥인식 성능시험용 DB 구성정보 ····· 43
표 3-7. 지정맥인식 성능시험용 DB 구성정보 ····· 43
표 3-8. 장정맥인식 성능시험용 DB 구성정보 ····· 44
표 3-9. 홍채인식 성능시험용 DB 구성정보 ····· 48

그림제목

그림 2-1. 정보보호산업진흥포털 누리집(“www.ksecurity.or.kr/bio”) …… 8
그림 3-1. 표준적합성 기능 선택에 따른 구현할 시나리오 …… 20

별표·별지

[별표 1] 시나리오 종류에 따른 구현 함수 …… 50
[별표 2] 테스트어서션 및 예시 …… 54
[별표 3] DLL 프로그램 예시(BioApi.cpp) …… 59
[별지 제1호 서식] 생체인식시스템 성능 시험・인증 신청서 …… 70
[별지 제2호 서식] 위조 생체인식 방어력 성능 시험・인증 신청서 …… 71
[별지 제3호 서식] 보안서약서 …… 72
[별지 제4호 서식] 인증연장 신청서 …… 73

제1장 생체인식시스템 성능 시험·인증 소개

1. 개요

- 한국인터넷진흥원(이하 “진흥원”이라 한다)은 생체인식시스템에 대한 경쟁력 강화와 수출 촉진을 위해 표준적합성, 알고리즘(SW) 성능, 위조 방어력(HW) 성능을 시험·평가 및 인증하는 국내 유일의 기관이다.

- 진흥원 내의 생체인식정보시험센터(Korea National Biometric Test Center, 이하 “K-NBTC” 이라 한다)에서 운영하고 있으며, 국내외 표준규격에 따라 시험을 실시하고 있다.

- 2006년에 K-NBTC를 개소하여, BioAPI 표준적합성 시험인증 서비스 제공을 시작으로 지문인식, 얼굴인식, 홍채인식, 정맥인식(손등정맥, 손가락정맥, 손바닥정맥) 알고리즘 성능 시험·인증과 위조 지문에 대한 방어력 성능 시험·평가 서비스를 제공하고 있다.

- 이를 통해 고성능의 생체인식 제품을 개발 가능하게 하여, 제품 경쟁력 향상의 기반이 될 수 있다.

- 또한, 진흥원의 생체인식시스템 성능시험은 산·학·연 전문가의 의견을 수렴해 개발된 기준과 절차로, 우수한 결과를 획득한 업체는 제품 판매시 우수 제품이라는 홍보를 할 수 있으며, 생체인식 제품 납품을 위한 입찰 경쟁시 진흥원의 성능 시험 결과보고서를 통해 품질의 우수성을 증명할 수 있다.

- 실제로 국내 정부기관 및 공공기관에 생체인식 제품 입찰 경쟁 시 진흥원의 생체인식 인증서를 제출을 하면 가산점을 부여하는 곳도 있다.

  ※ 생체인식시스템 성능 인증서 제출에 따른 가산점 부여는 필수사항은 아님

● 본 안내서는 생체인식 제품의 생체인식시스템 성능 시험·인증 절차와 방법 및 기준에 대해 설명한다.

※ 위조 생체인식 방어력 성능 시험·인증은 정보보호산업진흥포털 내의 물리보안 생체인식 제품시험·인증에 관한 페이지(“www.ksecurity.or.kr/bio”)의 “위조 생체인식 시스템 성능 시험·인증 안내서”를 참고

2. 진흥원 안내

● 진흥원의 생체인식시스템 성능 시험실 위치는 다음과 같다.

– 서울시 송파구 중대로 135 (가락동) IT벤처타워 서관 8층



진흥원의 생체인식 성능 시험 및 인증에 대한 자세한 사항은 누리집 또는 담당자 문의를 통해 확인 할 수 있다.

– 누리집 : http://www.ksecurity.or.kr/bio
– 담당자 이메일(서울) : knbtc@kisa.or.kr
– 담당자 연락처 : 02-405-6633

제2장 생체인식시스템 성능 시험·인증 절차

1. 시험 · 인증 개요

- K-NBTC에서는 생체인식 표준적합성 및 알고리즘 성능에 대해 시험 및 인증을 실시하며, 표 2-1과 같이 총 5단계로 구성된다.
- 1단계 시험 준비에서 마지막 5단계 결과보고서 및 인증서 발급까지 약 1.5개월 정도 소요된다.

표 2-1. 생체인식시스템 표준적합성 및 성능 시험 · 인증 절차

단계	절차		세부내용
1	시험준비		- (진흥원) 생체인식시스템 성능 시험·인증 관련 제반 절차 안내 - (신청 예정인) 시험 절차 문의
2	시험신청		- (신청인) 서류(시험 신청서, 보안서약서) 및 시험신청 공문 제출(이메일) - (신청인) 표준규격에 맞는 DLL 개발 및 필수 기능이 포함된 시험대상 제품 제출 - (진흥원) 신청서 접수 및 검토(제출물 보관)
3	사전 시험	표준 적합성	- (진흥원) 제출된 DLL 사전시험
		성능	- (신청인) 시험용 성능 DB 대상 사전시험
4	본 시험		- (신청인) 서류(시험 신청서, 보안서약서) 및 시험신청 공문 제출(이메일) - (진흥원) 신청서 접수 및 본 시험일 확정 - (신청인) 표준규격에 맞는 DLL 개발 및 제출 - (진흥원) 제출된 DLL 대상 본 시험 수행
5	인증		- (진흥원) 제품별 시험·평가 결과 기반 결과보고서 작성 - (진흥원) 본 시험 평가결과, 평가점수가 인증기준 통과 시 인증서 발급

※ '생체인식시스템 시험 · 인증'은 생체인식 산업 활성화를 위하여 현재 무료로 진행되고 있음

※ 진흥원에서 제공하는 시험용 DB는 K-NBTC에서만 사용 가능하며, 장비 반출 시 저장매체를 진흥원에 제출함(제출된 저장매체는 진흥원에서 폐기)

2. 시험 절차

1단계 (시험준비)

● 진흥원의 생체인식시스템 표준적합성 및 성능 시험·인증을 이용하려는 생체인식시스템 제품 제조·판매자(이하 “신청 예정인”이라고 한다)는 정보보호산업진흥포털 홈페이지 또는 K-NBTC 담당자에게 문의를 통해 시험, 인증 절차 및 방법에 관해 알아볼 수 있다.



≫ 그림 2-1. 정보보호산업진흥포털 누리집(“www.ksecurity.or.kr/bio”)

2단계 (시험신청)

- 신청 예정인은 생체인식 표준적합성 및 성능 시험·인증 시험을 신청하고자 할 때에는 아래의 서류를 진흥원에 제출하여야 하며, 제출방법은 전자우편(knbtc@kisa.or.kr)으로 가능하다.

  1. 사업자등록증
  2. 별지 제1호 생체인식 시스템 성능 시험·인증 신청서 또는 별지 제2호 위조 생체인식 방어력 성능 시험·인증 신청서
  3. 별지 제3호 보안서약서

- 신청 예정인은 진흥원에 시험 신청 후, 진흥원과 최종 시험일정을 협의하여 정해야 한다.

  ※ 시험 신청 수요 등에 따라 신청된 일정은 조정될 수 있음

- 신청 예정인은 신청 기간 내 필수기능이 탑재된 제품을 제출하며, K-NBTC 담당자를 통해 필수기능의 정상 작동 여부를 검증하고, 문제없을 시 신청 접수가 되며 신청인으로 전환된다.

- 신청인이 제출한 제품은 수정, 보완될 수 없으며, K-NBTC 담당자의 요청에 의해 특정한 경우에만 조치할 수 있다.

3단계 (사전시험)

- 시험 신청인은 사전시험 또는 연구목적으로 K-NBTC의 생체인식 DB를 이용할 수 있다.

- 사전시험은 자사의 제품이 표준적합성 및 성능시험의 인증기준을 통과할 수 있는지 여부를 사전에 확인하여 인증기준에 근접한지 여부를 판단하기 위함이다.

- 연구 목적이란, 자사의 생체인식 제품이 어느 정도 표준에 적합한지, 성능 수준은 얼마나 되는지를 파악하기 위해서 시험하는 경우이다.

- 신청인은 K-NBTC 담당자에게 K-NBTC Lab 사용을 요청하고, 해당 담당자는 사용 일정 조율 후 신청인에게 일정을 안내하며 신청인은 K-NBTC Lab에 방문하여 보유된 생체인식 DB를 대상으로 신청인 장비를 시험할 수 있다.

※ 생체인식 DB는 오직 K-NBTC Lab에서만 이용할 수 있으며, DB 정보를 유출하거나 무단 반출할 경우 법적인 처벌을 받을 수 있음

● 얼굴인식 사전 성능시험용 DB는 총 900장이며, 구성정보는 아래 표 2-2와 같다.

» 표 2-2. 사전 성능시험에 사용되는 얼굴인식 DB 구성정보

<table>
<tr><th>구 분</th><th>내 용</th></tr>
<tr><td>인원(명)</td><td>50명</td></tr>
<tr><td>구성
내용</td><td>
<table>
<tr><td>등록용DB</td><td colspan="4">50장</td></tr>
<tr><td rowspan="3">인식용DB</td><td>조명방향(8종류)</td><td>표정(4종류)</td><td>포즈(3종류)</td><td>액세서리(2종류)</td></tr>
<tr><td>400장(50*8)</td><td>200장(50*4)</td><td>150장(50*3)</td><td>100장(50*2)</td></tr>
<tr><td colspan="4">총 : 900장</td></tr>
</table>
</td></tr>
<tr><td>등록용</td><td>– 촬영조건 : 정면/주백색(형광등)/무표정/정면조명/정면포즈</td></tr>
<tr><td>인식용</td><td>– 조명방향(8종류): 0°, 45°, 90°, 135°, 180°, 225°, 270°, 315°
※ 0°(정면, 주광색/백열등), 45°~315°(주백색/형광등)
– 표정(4종류): 웃는 표정, 화난 표정, 눈감은 표정, 놀란 표정
– 포즈(3종류): 22.5°(Left, Right), 45°(Left)
– 액세서리(2종류): 헤어밴드, 안경 착용
※ 배경 : 블루스크린</td></tr>
<tr><td>형식</td><td>– 이미지크기(가로x세로) : 640×480
– 파일 포맷 : JPG 파일</td></tr>
</table>

● 지문인식 사전 성능시험용 DB는 각 센서 4개를 통해 생성한 DB가 구축되어 있으며, 총 DB 수는 3,000장으로 구성정보는 아래 표 2-3과 같다.

표 2-3. 사전 성능시험에 사용되는 지문인식 DB 구성정보

구 분 \ 센 서	IZZIX FD-100	UareU	EyeD Hamster	Magic Secure2500
인원(명)	25명	25명	25명	25명
1인 손가락(개) ※ 내용 : (오른손/왼손)엄지, 검지, 중지	6개	6개	6개	6개
손가락 당 영상(장) ※ 영상내용 : 품질차이	5장	5장	5장	5장
총 영상(장)	750장	750장	750장	750장
	총 3,000장			
가로 x 세로(Pixel)	280x320	292x525	248x292	288x320
파일포맷	RAW 파일			

● 홍채인식 사전 성능시험용 DB는 총 2,040장이며, 구성정보는 아래 표 2-4와 같다.

표 2-4. 사전 성능시험에 사용되는 홍채인식 DB 구성정보

구 분	내 용	
인원(명)	51명	
대상 눈	양쪽 눈(오른쪽 눈, 왼쪽 눈) ※ 안경착용 영상 미포함, 콘택트렌즈 착용 영상포함	
영상타입	A타입(정면 응시)	B타입(비스듬히 응시)
	홍채를 장치에 정면으로 응시한 상태에서 촬영	홍채를 장치에 5-10도 off-angle에서 촬영
총 영상(장)	51명×양쪽 눈×10장 = 1,020장	51명×양쪽 눈×10장 = 1,020장
	총 2,040장	
가로 x 세로(Pixel)	640x480	
파일포맷	BMP 파일	

● 정맥인식은 손등정맥인식, 손가락정맥인식(이하 '지정맥인식')과 손바닥정맥인식(이하 '장정맥인식')이 있으며, 손등정맥인식 사전 성능시험용 DB는 총 2,040장이며, 구성정보는 아래와 표 2-5와 같다.

표 2-5. 사전 성능시험에 사용되는 손등정맥인식 DB 구성정보

구 분	내 용	
인원(명)	51명	
대상 손	양 손(오른손, 왼손)	
영상타입	A타입(고정형)	B타입(움직임형)
	손을 장치에 고정한 상태에서 촬영	손을 장치에 넣었다 뺐다를 반복하여 촬영
총 영상(장)	51명×양 손×10장 = 1,020장	51명×양 손×10장 = 1,020장
	총 2,040장	
가로 x 세로(Pixel)	640x480	
파일포맷	BMP 파일	

● 지정맥인식 사전 성능시험용 DB는 총 4,080장이며, 구성정보는 아래 표 2-6과 같다.

표 2-6. 사전 성능시험에 사용되는 지정맥인식 DB 구성정보

구 분	내 용	
인원(명)	51명	
대상 손	양 손(오른손, 왼손) 검지, 중지	
영상타입	A타입(고정형)	B타입(움직임형)
	손가락을 장치에 고정한 상태에서 촬영	손가락을 장치에 넣었다 뺐다를 반복하여 촬영
총 영상(장)	51명×(양 손 검지, 중지) ×10장 = 2,040장	51명×(양 손 검지, 중지) ×10장 = 2,040장
	총 4,080장	
가로 x 세로(Pixel)	1,600x1,200	
파일포맷	BMP 파일	

● 장정맥인식 사전 성능시험용 DB는 총 2,400장이며, 구성정보는 아래 표 2-7과 같다.

표 2-7. 사전 성능시험에 사용되는 장정맥인식 DB 구성정보

구 분	내 용
인원(명)	40명
대상 손	양 손(오른손/왼손)
총 영상(장)	40명×(양 손)×30장 ※ 동일 손바닥의 위치 변화 및 압력에 따른 정맥 선명도 차이
	총 2,400장
가로 x 세로(Pixel)	512x512
파일포맷	BMP 파일

※ 장정맥 이미지는 AI 기술을 기반으로 제작되었으며, 시험 적합성 검증이 완료된 이미지만 활용

● 표준적합성 사전시험은 신청인이 '제3장 표준적합성 소개'에 기술한 내용에 따라 DLL 파일을 개발하여 진흥원에 제출하여야 하며, 제출된 DLL 파일은 시험기관인 K-NBTC에서 표준적합성 시험도구인 CTS(Conformance Test Suite)로 시험하여 그 결과를 아래 표 2-8과 같이 신청인에게 제공한다.

» 표 2-8. 표준적합성 사전시험 결과표

구 분	① Verification BSP	② Identification BSP	③ Capture BSP	④ Verification Engine	⑤ Identification Engine	시험 결과[1]
BioSPI_BSPLoad	○	○	○	○	○	
BioSPI_BSPUnload	○	○	○	○	○	
BioSPI_BSPAttach	○	○	○	○	○	
BioSPI_BSPDetach	○	○	○	○	○	
BioSPI_QueryUnits	○	○	○			
BioSPI_FreeBIRHandle	○	○	○	○	○	
BioSPI_GetBIRFromHandle	○	○	○	○	○	
BioSPI_GetHeaderFromHandle	○	○	○	○	○	
BioSPI_EnableEvents	○	○	○	○	○	
BioSPI_Capture			○			
BioSPI_CreateTemplate				○	○	
BioSPI_Process				○	○	
BioSPI_VerifyMatch				○	○	
BioSPI_IdentifyMatch					○	
BioSPI_Enroll	○	○				
BioSPI_Verify	○	○				
BioSPI_Identify		○				
BioSPI_Cancel	○	○	○	○	○	
BioSPI_Free	○	○	○	○	○	

1) 시험결과 : Pass 또는 Fail로 표기

- 신청인은 표준적합성 및 성능 사전시험에 대해 사전 일정 협의 후, 반드시 진흥원의 K-NBTC를 방문하여 직접 이를 수행해야 한다.
- 또한, 사전시험 단계를 건너뛰고, 본 시험 진행도 가능하지만, 사전시험을 수행하는 것이 본 시험에서 오류를 줄이고, 인증에 걸리는 시간을 단축할 수 있다.

4단계 (본 시험)

- 본 시험을 위해서는 신청인이 원하는 시험·인증 기술 분야에 따라 DLL파일을 개발 후, K-NBTC 담당자에게 제출하여야 한다.
- 표준적합성 및 성능의 각 기술 분야(지문, 얼굴, 정맥, 홍채)에 따른 DLL을 만드는 방법은 3장 평가분야에 기술되어 있는 내용을 준수하여야 한다.

5단계 (인증)

- 본 시험에서는 표준적합성의 경우 요구되는 필수 SPI가 모두 제대로 구현되어야 하며, 지문인식과 얼굴인식, 정맥인식, 홍채인식 성능의 경우는 아래 표 2-9에서 요구하는 인증기준을 통과해야 한다.
- 지문인식과 얼굴인식에서 사용되는 인증기준은 EER(Equal Error Rate)을 사용하며, FTE(Failure to Enroll Rate), FTA(Failure to Acquire Rate) 값을 참고한다.
- EER은 FMR(False Match Rate)과 FNMR(False Non-Match Rate)이 일치하는 에러값을 의미하므로, FMR, FNMR을 먼저 도출한 후, EER을 계산한다. FTE, FTA는 영상 DB의 이미지를 정상적으로 읽지 못한 것으로, 5% 이상이 되면, 시험결과가 유효하지 않은 것으로 판명한다.
- 결과값에 따라 생체인식시스템의 성능 인증이 결정되므로, 이를 인증기준이라고 한다. 인증기준은 진흥원 내부 전문가로 구성된 인증위원회를 통해 결정되며, 시장 상황 및 표준변화에 따라 그 값이 변화할 수 있다.

표 2-9. 생체인식 성능 인증기준

분야	EER			
지문인식	1%이하(센서 별로 구축한 4개의 DB중 3개의 DB가 1%이하여야 함)			
얼굴인식	조명	표정	포즈	액세서리
	1%이하	2%이하	2%이하	1.5%이하
손등정맥인식	A타입(고정형)		B타입(움직임형)	
	1%이하		1.5%이하	
지정맥인식	A타입(고정형)		B타입(움직임형)	
	1%이하		1.5%이하	
장정맥인식	1.5%이하			
홍채인식	A타입(정면 응시)		B타입(비스듬히 응시)	
	1%이하		1.5%이하	

● EER 및 관련 값의 자세한 내용은 아래 표 2-10와 같다.

표 2-10. 성능 시험항목 계산식

항 목	세부내용
FMR	○ 타인 수락율 - (정의) 생체인식 알고리즘이 2개의 이미지를 비교하여, 동일한 이미지로 판별했지만, 실제로는 상이한 이미지일 경우의 비율 -(계산) 서로 다른 이미지를 비교하는 타인의 시도(NI, Number of Impostor Recognition Attempts)에 대해 임계 값(t) 이상으로 나타나는 정합 함수(sim, Impostor Matching Score)를 가진 타인의 비율
FNMR	○ 본인 거부율 - (정의) 생체인식 알고리즘이 2개의 이미지를 비교하여, 상이한 이미지로 판별했지만, 실제로는 동일한 이미지일 경우의 비율 - (계산) 서로 동일한 이미지를 비교하는 본인의 시도(NG, Number of Genuine Recognition Attempts)에 대해 어떤 임계 값(t) 미만으로 나타나는 정합 함수(sgm, Genuine Matching Score)를 가진 본인과 본인의 등록 거부 및 비교 거부 회수의 합(REJNG)의 비율
EER	- FMR과 FNMR이 같을 때의 Error Rate
FTE	- 등록용 영상 DB에서 영상 이미지를 읽어 들이지 못하거나, 특징량을 추출하지 못한 경우의 비율
FTA	- 인식용 영상 DB에서 영상 이미지를 읽어 들이지 못하거나, 특징량을 추출하지 못한 경우의 비율

- K-NBTC에서는 결과값(EER · FMR · FNRM · FTA · FTE)의 유효성 및 인증기준 적합여부를 확인하고 합격 기준을 통과하면 인증서를 발급한다.

  ※ 얼굴인식은 전체 4종(조명, 표정, 포즈, 액세서리)에서 일부(1~3종) 통과 시 부분인증이 가능하며, 이 경우 통과된 분야를 인증서에 명시한다.

  ※ 손등정맥인식, 지정맥인식, 홍채인식의 경우, A타입 1종만 성능 인증기준 통과 시 A타입에 대해서만 부분인증이 가능하다. (A타입 부분인증의 경우 A타입과 B타입의 EER 값의 차이가 5% 이하여야 한다.)

  ※ 장정맥인식의 경우, 본 시험 DB 전체에서 이미지를 정상적으로 인식하지 못한 EER 값이 1.5% 이하일 경우만 인증을 받을 수 있다.

- 생체인식시스템 표준적합성 및 성능에 대한 인증 유효기간은 인증서가 발급된 날로부터 3년이다.

- 인증 유효기간은 기존 인증 받은 모듈의 수정사항이 없을 경우에 한하여, 1회에 한해 2년 간 유효기간을 연장할 수 있다.

- 이를 위해서는 반드시 최초 인증 유효기간이 만료되기 전에 별지 제4호 인증연장 신청서를 작성하여 진흥원에 제출해야 한다.

  ※ 단, 진흥원의 성능시험용 DB 또는 인증기준이 변경되었을 경우에는 변경된 내용을 적용하여 재시험 후, 그 결과에 따라 인증 연장 여부를 결정한다.

제3장 평가분야

1. 표준적합성 시험

(1) 개요

- 표준적합성 평가란, 바이오인식 제품의 BioAPI BSP(Biometrics Service Provider) 모듈이 ISO/IEC 19784-1에서 정의한 BioAPI 표준 규격에 맞추어 구현되었음을 검증하기 위한 시험이며, 시험기술에 대한 표준은 ISO/IEC 24709-1에 포함되어 있다.
  시험방법은 ISO/IEC 24709-2에 기술된 시험방법을 따른다.

- 즉, 신청인은 표준적합성 시험을 위해서는 BioAPI라는 기본함수에 대한 입출력 구현내용을 알아야 하며, 이는 ISO/IEC 19784-1의 9절 BioAPI Service Provider Interface에 자세히 기술되어 있으므로, 이를 참고해야 하며, 24709-1, 24709-2도 구현 시 함께 참고한다.

- ISO/IEC 문서는 한국표준정보망(http://www.kssn.net)에서 유료로 구입할 수 있으며, K-NBTC에서는 이를 어떠한 형태로든 제공하지 않는다.

(2) 표준적합성 평가 대상

- 표준적합성은 BSP 제품이 제공하는 기능에 따라 Verification, Identification 2가지 기능에 대하여 시험한다. (아래 표 3-1 참조)

표 3-1. BSP 제공 기능에 따른 시험 시나리오

구분 / 제공 기능	시험 시나리오명	관련내용
Verification	Verification BSP	3.1절
	Capture BSP	3.3절
	Verification Engine	3.4절
Identification	Identification BSP	3.2절
	Capture BSP	3.3절
	Identification Engine	3.5절

- Verification일 경우에는 Verification BSP, Capture BSP, Verification Engine 총 3가지의 시나리오를 준비해야 하며, 이에 대한 설명은 3.1, 3.3, 3.4절에 각각 기술되어 있다.
- Identification일 경우에 Identification BSP, Capture BSP, Identification Engine 총 3가지의 시나리오를 준비해야 하며, 이에 대한 설명은 3.2, 3.3, 3.5절에 각각 기술되어 있다.

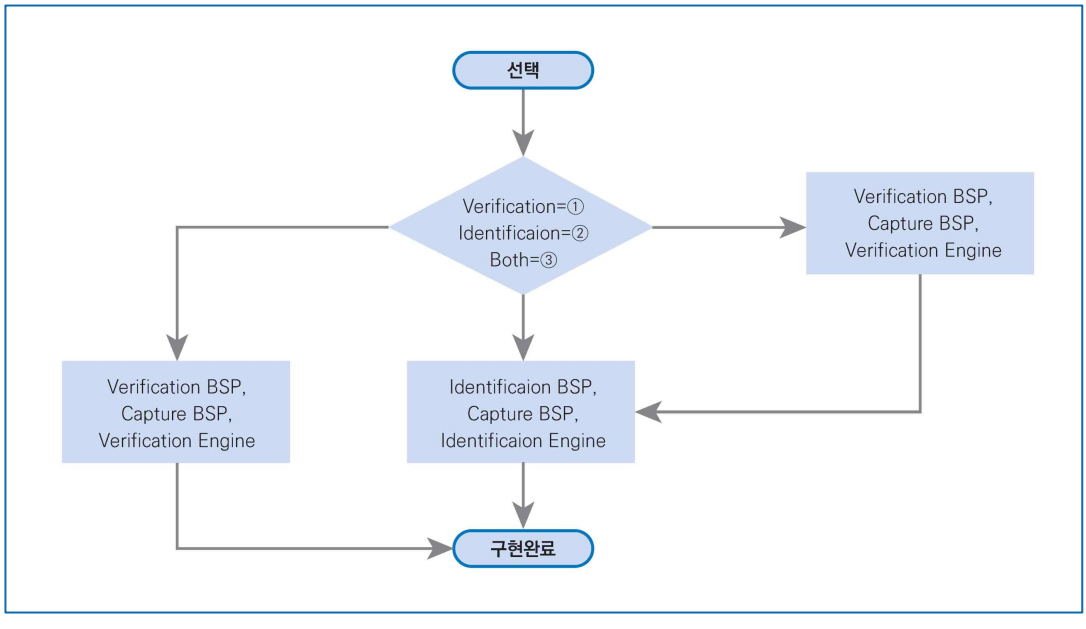


그림 3-1. 표준적합성 기능 선택에 따른 구현할 시나리오

(3) 시나리오 종류

각 시나리오에 따라 구현해야 하는 함수가 존재하며, 별표 1 시나리오 종류에 따른 구현 함수를 통해 세부 목록을 확인할 수 있다.

- 시나리오① : Verification BSP
  - BSP 또는 어플리케이션에서 영상을 획득하고 저장된 템플릿과 비교하는 BioSPI Enroll 및 BioSPI_Verify 함수를 대상으로 시험한다. 13개의 SPI를 필수 함수로 구현한다.

- 시나리오② : Identification BSP
  - Verification BSP와 동일하게 BSP 또는 어플리케이션에서 영상을 획득한 후 1:1 비교가 아닌 저장된 다수의 템플릿과 비교하는 BioSPI_Enroll 및 BioSPI_Verify 함수를 대상으로 시험한다. 14개의 SPI를 필수 함수로 구현해야 한다.

- 시나리오③ : Capture BSP
  - 생체정보 획득을 위한 BioSPI_Capture 함수를 대상으로 시험한다. 12개의 SPI를 필수 함수로 구현해야 한다.

- 시나리오④ : Verification Engine
  - 생체정보를 획득하여 템플릿을 생성하고 BioSPI_Process를 통해 영상을 처리한 후 검증하기 위한 비교는 BioSPI_VerifyMatch를 통해 이루어지며, BioSPI_CreateTemplate, BioSPI_ Process, BioSPI_VerifyMatch 함수를 대상으로 시험한다. 13개의 SPI를 필수 함수로 구현해야 한다.

- 시나리오⑤ : Identification Engine
  - 프로세스는 Verification Engine BSP와 동일하나 BioSPI_VerifyMatch를 통해 1:N 비교가 수행되며, BioSPI_CreateTemplate, BioSPI_Process, BioSPI_VerifyMatch 함수를 대상으로 시험한다. 14개의 SPI를 필수 함수로 구현해야 한다.

(4) SPI(Service Provider Interface)

● SPI 함수 기능

표 3-2. SPI 함수 기능

SPI Component Management Functions	
BioSPI_BSPLoad	BioAPI와 BSP간의 컴포넌트를 초기화 함
BioSPI_BSPUnload	BioAPI_BSPLoad에서 수행된 초기화를 되돌림
BioSPI_BSPAttach	어플리케이션에 의해 예상되었던 BSP시스템의 버전과 적합하다는 것을 검증하기 위해 프레임워크에 의해 실행
BioSPI_BSPDetach	모든 BSP의 연결 해제
BioSPI_QueryUnits	모든 BioAPI유닛에 대한 정보를 제공함
Handle Functions	
BioSPI_FreeBIRHandle	특정한 BIR 핸들과 관련된 메모리와 자원 해제
BioSPI_GetBIRFromHandle	특정한 BIR 핸들 얻음
BioSPI_GetHeaderFromHandle	핸들로부터 헤더정보 반환
Callback and Event Functions	
BioSPI_EnableEvents	이벤트 mask에 의해 지정된 이벤트 활성화
Biometric Functions	
BioSPI_Capture	센서로부터 데이터(생체정보) 입력받음
BioSPI_CreateTemplate	새로운 등록 템플릿을 생성할 목적으로 BIR을 취함
BioSPI_Process	Capture를 통해 획득된 데이터를 검증, 식별위해 처리
BioSPI_VerifyMatch	두 개의 BIR간에 검증 수행
BioSPI_IdentifyMatch	1:1 식별 매칭
BioSPI_Eroll	검증을 위한 사용자 등록과정 수행
BioSPI_Verify	데이터를 추출하고 참조 템플릿과 비교
BioSPI_Identify	1:N 식별 매칭
Utility Functions	
BioSPI_Cancel	BSPHandle과 관련된 함수호출을 취소
BioSPI_Free	메모리 블록을 해제함

(5) 테스트어서션

테스트어서션 종류

- BioAPI BSP 표준적합성 평가의 대상이 되는 것은 BioSPI 함수들 중에서 14가지 주요 함수이며, 총 50개의 테스트어서션(별표2 테스트어서션 및 예시 참고)이 해당된다.
- 테스트어서션은 각 함수의 입/출력 검증, 파라미터 및 리턴 값들을 명확하게 구분하기 위해 각 필수 함수에 대한 가정으로, 실질적인 표준적합성 시험은 시나리오에 기반을 둔 테스트어서션을 통해 수행하게 된다.
- 테스트어서션의 ID는 1a, 1b, 2a, 2b처럼 숫자와 알파벳으로 표현되는데, 숫자는 테스트 대상이 되는 BioSPI 함수의 종류를 의미하고 알파벳은 테스트 종류를 나타낸다.

테스트어서션 파라미터

- 테스트어서션은 XML로 작성된 일종의 테스트 실행 스크립트이다. CTS 평가 도구는 테스트어서션 분석기능과 실행기능을 포함하고 있어서, 테스트어서션(별표 2 테스트어서션 및 예시 참고)을 실행함으로써 BSP 제품의 표준적합성 시험을 진행한다.
- ISO/IEC 24709-2:2007에 기술된 테스트어서션들의 입력 파라미터 상당 부분은 중복되거나 또는 시스템 내부적으로 처리할 수 있다. 그러나, BSP 제품의 'UUID' 같은 정보는 BSP 제품의 특성에 기반하기 때문에 K-NBTC에 정보를 제출하여야 한다.

평가 수행

- 본 기관의 표준적합성 평가도구인 CTS는 각 테스트어서션을 읽어서 실행하는 일종의 인터프리터로 볼 수 있다.

- 각 테스트어서션의 실행결과는 PASS/ FAIL/ UNDECIDED로 구분된다. PASS는 해당 테스트항목에 대하여 BSP 제품이 제대로 동작하였음을 의미하며, FAIL은 BSP제품이 제대로 기능이 동작하지 않거나 예상되는 응대(response)를 하지 않았음을 의미한다. UNDECIDED는 테스트 대상이 되는 기능이 아닌 다른 부분에서 문제가 발생하여 테스트를 제대로 수행할 수 없음을 나타낸다.

- 평가대상인 BSP 제품이 표준적합성 테스트를 통과하기 위해서는 테스트어서션 목록에 포함된 모든 테스트어서션들의 수행결과가 PASS가 되어야 한다. 하나라도 그 결과가 FAIL이나 UNDECIDED가 된다면 표준적합성 테스트를 통과하지 못함을 의미한다.

(6) 주의사항

- 초기에 표준적합성 시험을 받은 BSP제품들에서 payload와 관련된 테스트어서션(예: T.A. 14b)을 통과하지 못하는 경우가 자주 발생하였다. 공통적으로 payload를 지원하지 않는 BSP 제품들로, 테스트어서션 파라미터인 _payloadSupported가 false로 지정되었다.

- Payload를 지원하지 않는 BSP제품의 경우에도 테스트어서션을 통과할 수 있도록 작성 되었음에도 이런 현상이 발생한 것에 대하여 그 원인을 살펴보면, payload를 지원하지 않는 BSP 제품이 표준적합성 시험을 통과하기 위해서는 몇 개의 BioSPI 함수들의 동작이 ISO/IEC 24709-2에 기술된 테스트케이스에 맞도록 동작해야 한다. 만일 이와 다르게 구현이 되어있다면 국제 표준적합성 시험을 통과하지 못한다.

- 테스트어서션 통과에 문제가 되는 BioSPI함수들은 BioSPI_Enroll(), BioSPI_CreateTemplate(), BioSPI_Verify() 이다. 테스트어서션을 통과하기 위하여 이들 함수가 특정 상황에서 반응해야 하는 내용은 다음과 같다.

BioSPI_Enroll(), BioSPI_CreateTemplate()

- 상황 : payload를 지원하지 않는 BSP모듈에 있는 위 함수들을 호출할 때, NULL 값이 아닌 payload 값을 인자로 넣어 호출하는 상황이다.

- 동작 : payload를 지원하지 않는데 payload 인자 값이 NULL이 아니라면 함수 종료시 에러메시지를 반환해야 한다. 그러나, 에러메시지 반환과는 별도로 함수의 작업은 계속 진행하여 newTemplate값을 생성(양수값) 하여야 한다. 즉, 작업을 진행하여 newTemplate값을 생성한 후 함수를 종료할 때 에러메시지를 반환한다.

- 반환값 : __BioAPIERR_BSP_UNABLE_TO_STORE_PAYLOAD

BioSPI_Verify()

- 상황: payload를 지원하지 않는 BSP모듈의 이 함수를 호출할 때, NULL 값이 아닌 payload 값을 인자로 넣어 호출하는 상황이다.

- 동작: payload를 지원하지 않는데 payload 인자값이 NULL이 아니라면 이를 무시하고 작업을 진행한다. 다른 특별한 문제가 없다면 _BioAPI_OK를 반환한다. 즉, 위 BioSPI_ Enroll()함수같이 작업을 진행하는 것은 동일하지만, 함수 종료시 __BioAPI_OK를 반환하는 점이 다르다.

- 반환값 : __BioAPI_OK

2. 지문인식 성능시험

(1) 개요

- 지문인식 성능시험을 진행하기 위해서는 아래 절에서 설명하는 윈도우 환경에서 동작하는 DLL(Dynamic Linking Library)을 구현해서 제출하여야 한다. K-NBTC 담당자는 신청인의 DLL로 지문인식 성능시험을 하게 되며, 시험결과가 인증기준을 통과하면, 진흥원에서는 인증서를 발급한다.

(2) DLL 개발 방법

- 신청인이 개발할 DLL은 다음과 같은 입출력을 가지는 3가지 함수를 반드시 내장하여야 한다. 이 3가지 함수가 성능시험도구의 인터페이스가 된다.

(1) KISA_Enroll

- KISA_Enroll는 지문 이미지를 읽어서 추출한 특징량을 저장하는 함수이며, 특히 등록용 이미지를 읽고, 이로부터 특징량을 추출/저장하는 함수이다. 이를 줄여서 "등록용 특징량 추출 함수"라고 한다.

  – 등록용 특징량 추출함수

```
int KISA_Enroll(
IN int iType,                    // 아래 pImage의 센서 종류(아래 표 3-3 참조)
IN unsigned char* pImage,        // 센서 종류에 따른 등록용 지문 이미지 픽셀 주소
OUT unsigned char* pEnroll,          // 추출된 특징량이 저장되는 메모리 주소
OUT int* piEnrollSize,           // 추출된 특징량 크기 (1,000,000bytes 보다 작아야함)
OUT int* piErrorCode,            // 신청인이 지정하는 에러 코드
OUT char arErrorMessage[256]     // 신청인이 지정하는 에러 메시지 )
```

  리턴 값은 성공시 1, 실패시 0을 리턴하면 된다.

표 3-3. 지문인식 센서 종류

구분	센서 종류	회사명	iType	사이즈(W*H)	메모리 크기
DIG00	IZZIX FD-100	디젠트	5	280*320	89,600 bytes
PER00	UareU	Digital Persona	7	292*525	153,300 bytes
NIT00	EyeD Hamster	니트젠	8	248*292	72,416 bytes
HUN00	MagicSecure2500	휴노	9	288*320	92,160 bytes

- KISA_Enroll 함수의 변수에 대한 설명은 아래와 같다.
  - (iType) 표 3-3에 나와 있는 센서의 종류를 나타낸다. 즉, iType이 5이면, IZZIX FD-100 센서를 나타내며, 해당 센서를 통해 지문을 촬영했다는 것을 의미한다. 센서별로 지문 이미지의 크기는 상이하므로, 입력되는 iType에 따라, 지문 이미지 RAW 파일을 추출해야 한다.
  - (pImage) 지문 이미지의 픽셀 주소이다. 이때, 이 주소는 지문 이미지(RAW파일)의 헤더부분을 스킵(skip)한 상태이다. 그러므로, pImage의 첫번째 바이트를 나타내는 (pImage+0)은 지문 이미지의 좌상단 첫번째 픽셀 주소를 가리킨다.
    또한, 지문 이미지의 한 픽셀은 1 byte 메모리 크기를 갖는다.
    예를 들어, DIG00의 경우 표 3-3에 280*320(W*H)으로 표기되어 있으므로 총 89,600 바이트의 크기를 지니게 되며, *(pImage+89601)과 같은 표기는 버퍼 오버플로우 에러를 발생시키며 이는 성능지표상 FTE 수치를 증가시킨다.
    그러므로, 신청인이 메모리 크기를 준수하지 않는 경우, 버퍼 오버플로우 에러가 발생할 수 있으므로 이에 주의한다.
  - (pEnroll) 신청인의 알고리즘이 지문 이미지에서 특징량을 추출한 후, 저장하는 특징량 데이터의 메모리 주소값을 의미한다. 이때 메모리 크기는 1,000,000바이트를 넘어서는 안된다.
  - (piEnrollSize) 신청인이 추출한 지문 이미지 특징량 데이터의 메모리 사이즈를 의미한다.
  - (piErrorCode, arErrorMessage) 신청인이 임의로 지정하는 에러 코드 및 메시지를 나타낸다. 에러인 경우는 KISA_Enroll 함수에서 0이 리턴되면, 신청인이 지정한 piErrorCode와 arErrorMessage에 에러 코드 및 에러 메시지를 시험도구에서 출력한다. 이 경우 FTE에 영향을 미치므로 신청인은 이를 주의 깊게 처리하도록 한다.

(2) KISA_Attempt

- KISA_Attempt는 지문 이미지를 읽어서 추출한 특징량을 저장하는 함수이며, 특히 인식용 이미지를 읽고, 이로부터 특징량을 추출/저장하는 함수이다. 이를 줄여서 “인식용 특징량 추출 함수”라고 한다.
- 인식용 특징량 추출 함수인 KISA_Attempt는 상기 KISA_Enroll과 함수의 기능 및 변수의 의미가 완전히 동일하다.

  – 인식용 특징량 추출함수

```
int KISA_Attempt(
IN int iType,                    // pImage의 센서 종류 (표 3-3 참조)
IN unsigned char* pImage,        // 센서 종류에 따른 인식용 지문 이미지 픽셀 주소
OUT unsigned char* pAttempt,     // 추출된 특징량이 저장되는 메모리 주소
OUT int* piAttemptSize,          // 추출된 특징량 크기
OUT int* piErrorCode,            // 신청인이 지정하는 에러 코드
OUT char arErrorMessage[256]     // 신청인이 지정하는 에러 메시지
)
```

리턴 값은 성공시 1, 실패시 0을 리턴하면 된다.

(3) KISA_Compare

- KISA_Compare는 KISA_Enroll의 pEnroll과 KISA_Attempt의 pAttempt에 각각 저장된 지문 이미지의 특징량을 비교하여, 정합점수를 도출하는 함수이다. 이를 줄여서 “비교용 함수”라고 한다.

– 비교용 함수

```
int KISA_Compare(
IN unsigned char* pEnroll,
                                    // KISA_Enroll 함수에서 넘긴 등록용 특징량 저장 주소
IN unsigned char* pAttempt,
                                    // KISA_Attempt 함수에서 넘긴 인식용 특징량 저장 주소
OUT double* pdScore,                //  0(타인지문)에서 1(동일지문)사이의 정합 점수
OUT int* piErrorCode,               // 신청인이 지정하는 에러 코드
OUT char arErrorMessage[256]     // 신청인이 지정하는 에러 메시지
)
```

- KISA_Compare 함수의 변수에 대한 설명은 아래와 같다.
  - (pEnroll) KISA_Enroll에서 추출된 등록용 지문 이미지 특징량이 저장된 메모리 주소값이다.
  - (pAttempt) KISA_Attempt에서 추출된 인식용 지문 이미지 특징량이 저장된 메모리 주소값이다.
  - (pdScore) 신청인 측 알고리즘이 pEnroll과 pAttempt를 비교하여 계산한 정합 점수이다. 반드시 0(타인지문)과 1(동일지문)사이의 값이어야 하며, 최대 소수점 3째자리 까지 그 값을 나타낼 수 있어야 한다. 만일 그 이외의 값을 지정할 경우 비교 거부로 판정하여 FNMR값에 영향을 미칠 수 있음에 주의한다.
  - (piErrorCode, arErrorMessage) 신청인이 임의로 지정하는 에러 코드 및 메시지를 나타낸다. 에러인 경우는 KISA_Compare 함수에서 0이 리턴되면, 신청인이 지정한 piErrorCode와 arErrorMessage에 에러 코드 및 에러 메시지를 시험도구에서 출력한다. 이 경우 FNMR에 영향을 미치므로 신청인은 이를 주의 깊게 처리하도록 한다.

(3) 성능시험용 데이터베이스

- 지문인식 성능시험용 DB는 사전시험용 성능 DB 구성정보와 동일하나, DB 크기가 더 크다.

표 3-4. 지문인식 성능시험용 DB 구성정보

구 분 \ 센 서	IZZIX FD-100	UareU	EyeD Hamster	Magic Secure2500
인원(명)	250명	250명	250명	250명
인원당 손가락(개) ※ 내용 : (오른손/왼손)엄지, 검지, 중지	6개	6개	6개	6개
손가락 당 영상(장) ※ 영상내용 : 품질차이	5장	5장	5장	5장
총 영상(장)	7,500장	7,500장	7,500장	7,500장
	총 30,000장			
가로 x 세로(Pixel)	280x320	292x525	248x292	288x320
파일포맷	RAW 파일			

(4) DLL 구현 예제 설명

- 본 절에서는 DLL 개발을 위한 구현 예제를 설명하며, 예제에 사용되는 파일은 홈페이지 (www.ksecurity/bio)의 자료실에서 다운로드할 수 있다.

- 다운로드 한 IndustryAlgorithm(지문) 폴더에서
  VC++ 6 사용자는 프로젝트 파일 IndustryAlgorithm_vc6.dsw,
  VC++ 2005 사용자는 프로젝트 파일 IndustryAlgorithm_vc2005.sln,
  VC++ 2010 사용자는 프로젝트 파일 IndustryAlgorithm_vc2010.sln을 선택하고, 이어서 관련 파일 중 BioApi.cpp를 선택한다.
  핵심 DLL 인터페이스 함수가 구현되어 있으며, 별표 3 DLL 프로그램 예시(BioApi.cpp)를 참고하면 된다.

(5) 주의사항

- 신청인이 제출한 DLL 파일 이외에 부속 파일들(dll, exe)을 포함할 경우, 시험도구 내에서 DLL이 로드되어 실행될 때 부속 파일들을 로드하지 못하고, 실행이 되지 않는 문제가 발생할 수 있다.
- 이는 시험도구 프로그램의 위치가 피평가 모듈 파일들의 위치와 다른 경로상에 존재하기 때문이다. 이를 해결하기 위해서는 기본적으로 피평가 모듈 내에서 부속 파일들을 접근할 때 현재 실행되는 프로그램(시험도구)의 위치에서 찾지 말고, 피평가 모듈의 위치에서 찾도록 구현되어야 한다.
- 부속 DLL이나 별도의 부속 파일을 이용하는 경우, 부속 파일들은 DLL과 동일한 경로상에 위치해야 하며, DLL의 경로를 얻는 방법은 별표 3 DLL 프로그램 예시(BioApi.cpp)를 참고하면 된다.

3. 얼굴인식 성능시험

(1) 개요

● 얼굴인식 성능시험을 위해서도 아래 절에서 설명하는 윈도우용 DLL(Dynamic Linking Library)을 구현해서 제출하여야 한다. K-NBTC 담당자는 신청인의 DLL로 얼굴인식 성능시험을 하게 되며, 시험결과가 인증기준을 통과하면, 진흥원에서는 인증서를 발급한다.

(2) DLL 개발 방법

● 신청인이 개발할 DLL에는 다음과 같은 입출력을 가지는 7가지 함수를 반드시 내장하여야 한다. 이 7가지 함수가 성능시험 도구의 인터페이스가 된다.

(1) KISA_Init

● KISA_Init 함수는 전체 모듈을 초기화하는 기능을 담당한다. 따라서 신청인은 KISA_Init 함수에서 생체인식장치를 초기화하거나 변수설정이나 기타 초기화를 완료하도록 구현하여야 한다. 함수의 선언은 다음과 같다.

- KISA_Init 함수

```
KISADLL BioAPI_RETURN  KISA_Init (
        const BioAPI_VERSION       *Version,
        uint32 Reserved1,
        const void                 * Reserved2,
        uint32                     * Reserved3,
        const void                 * Reserved4
)
```

- Parameters는 모두 사용되지 않는다. 모두 0또는 NULL로 설정하면 된다.
- 리턴값은 성공하면 BioAPI_OK를, 실패하면 그 이외의 값을 반환한다.

(2) KISA_Terminate

- 생체인식 모듈을 종료하는 함수이다. 따라서 신청인은 KISA_ Terminate 함수에서 생체인식장치를 종료하거나 내부 메모리 반환 등과 같은 종료작업을 완료하도록 구현하여야 한다. 함수의 선언은 다음과 같다.

  – KISA_Terminate 함수

```
KISADLL BioAPI_RETURN  KISA_Terminate ( )
```

  – Parameters 없음

  – 리턴값은 성공하면 BioAPI_OK를, 실패하면 그 이외의 값을 반환한다.

(3) KISA_FreeBIRHandle

- 이 함수는 핸들(Handle)에 연결된 BioAPI_BIR 메모리를 해제한다. 더 이상 BioAPI_BIR 데이터가 사용되지 않을 경우 호출한다.

  – KISA_FreeBIRHandle 함수

```
KISADLL BioAPI_RETURN  KISA_FreeBIRHandle (
        BioAPI_HANDLE              ModuleHandle,
        BioAPI_BIR_HANDLE          Handle
        )
```

  – Parameters는 아래와 같다.

  - ModuleHandle : 0으로 설정한다.
  - Handle : 메모리를 해제하고자 하는 핸들이다.

  – 리턴값은 성공하면 BioAPI_OK를, 실패하면 그 이외의 값을 반환한다.

(4) KISA_GetBIRFromHandle

● 이 함수는 핸들(Handle)로부터 BioAPI_BIR 데이터를 반환한다.

– KISA_GerBIRFromHandle 함수

```
KISADLL BioAPI_RETURN  KISA_GetBIRFromHandle (
BioAPI_HANDLE          ModuleHandle,
BioAPI_BIR_HANDLE      Handle,
BioAPI_BIR_PTR         *BIR
)
```

– Parameters는 아래와 같다.

- ModuleHandle : 0으로 설정한다.
- Handle : 데이터를 가져오고자 하는 Handle이다.
- BIR (output) : 반환되는 BioAPI_BIR 데이터이다.

– 리턴값은 성공하면 BioAPI_OK를, 실패하면 그 이외의 값을 반환한다.

(5) KISA_Enroll

● KISA_Enroll은 얼굴 이미지를 읽어서 추출한 특징량을 저장하는 함수이며, 특히 등록용 이미지를 읽고, 이로부터 특징량을 추출/저장하는 함수이다. 이를 줄여서 “등록용 특징량 추출 함수”라고 한다.

– KISA_Enroll 함수

```
KISADLL BioAPI_RETURN  KISA_Enroll (
        BioAPI_HANDLE              ModuleHandle,
        const BioAPI_INPUT_BIR     *CapturedBIR,
        const BioAPI_INPUT_BIR     *StoredTemplate,
        BioAPI_BIR_HANDLE_PTR      NewTemplate,
        const BioAPI_DATA          *Payload
)
```

- 리턴값은 성공하면 BioAPI_OK를 반환하며, 실패시 이외의 값을 반환한다.

  – Parameters는 아래와 같다.

  - (CapturedBIR) 얼굴인식 이미지의 정보가 저장되며, 이미지 파일의 메모리 주소값인 시작 포인터는 아래와 같은 코드로 얻어온다.
    BioAPI_BIR_PTR pBIR = CapturedBIR→InputBIR.BIR;
    pBIR에 이미지 정보를 얻어오면 pBIR→Signature→Data에 이미지 파일의 시작 포인터를 저장한다.
  - (NewTemplate (output)) 등록용 이미지로부터 얼굴 이미지의 특징량 데이터를 저장하며, 아래와 같은 코드를 이용한다.
    uint8 Enroll[1000] = "5518bfe3fec4dbce392ddd4482812f7b";
    //특징량 데이터를 "5518bfe3fec4dbce392ddd4482812f7b"라고
    //가정하고, Enroll에 저장한다.
    BioAPI_HANDLE Handle = AddNewBIR(Enroll, 1000);
    *NewTemplate = Handle;
  - (StoredTemplate) NewTemplate에 저장된 특징량 데이터가 저장되는 변수로, 진흥원 시험도구에서 자동으로 저장된다.
  - 나머지 Parameters는 무시한다.

(6) KISA_Attempt

- KISA_Attempt는 얼굴 이미지를 읽어서 추출한 특징량을 저장하는 함수이며, 특히 인식용 이미지를 읽고, 이로부터 특징량을 추출/저장하는 함수이다. 이를 줄여서 “인식용 특징량 추출 함수”라고 한다.
- 인식용 특징량 추출 함수인 KISA_Attempt는 상기 KISA_Enroll과 함수의 기능 및 변수의 의미가 완전히 동일하다.

  – KISA_Attempt 함수

```
KISADLL BioAPI_RETURN  KISA_Attempt
(
        BioAPI_HANDLE                   ModuleHandle,
        const BioAPI_INPUT_BIR          *CapturedBIR,
        const BioAPI_BIR_HANDLE_PTR     ProcessedBIR,
)
```

- 리턴값은 성공하면 BioAPI_OK를, 실패하면 그 이외의 값을 반환한다.

  – Parameters는 아래와 같다.
  - (CapturedBIR) 얼굴인식 이미지의 정보가 저장되며, KISA_Enroll의 CapturedBIR과 동일한 기능을 한다.
  - (ProcessedBIR (output)) 얼굴인식 이미지의 특징량 데이터를 저장하며, KISA_Enroll의 NewTemplate과 동일한 기능을 한다.
  - 나머지 Parameters는 무시한다.

(7) KISA_Compare

- KISA_Compare는 KISA_Enroll의 StoredTemplate과 KISA_Attempt의 ProcessedBIR에 각각 저장된 얼굴 이미지의 특징량을 비교하여, 정합점수를 도출하는 함수이다.

  – KISA_Compare 함수

```
KISADLL BioAPI_RETURN  KISA_Compare (
    BioAPI_HANDLE           ModuleHandle,
    const BioAPI_FAR        *MaxFARRequested,
    const BioAPI_FRR        *MaxFRRRequested,
    const BioAPI_BOOL       *FARPrecedence,
    const BioAPI_INPUT_BIR  *ProcessedBIR,
    const BioAPI_INPUT_BIR  *StoredTemplate,
    BioAPI_BIR_HANDLE       *AdaptedBIR,
    BioAPI_BOOL             *Result,
```

```
        BioAPI_FAR_PTR FARAchieved,
        BioAPI_FAR_PTR FRRAchieved,
        BioAPI_DATA_PTR *Payload )
```

- 리턴값은 성공하면 BioAPI_OK를 반환하며, 실패시 이외의 값을 반환한다.
  - Parameters는 아래와 같다.
    - (StoredTemplate) KISA_Enroll 함수를 통하여 입력했던 특징량은 StoredTemplate→BIRinBSP에 들어있다.
    - (ProcessedBIR) KISA_Attempt 함수를 통하여 입력했던 특징량은 ProcessedBIR→BIRinBSP에 들어있다.
    - (Result (output)) 두 이미지 사이의 유사도를 정수로 표현한다.(0~100사이)
    - 나머지 Parameters는 무시한다.

(3) 성능시험용 데이터베이스

● 얼굴 인식 성능시험용 데이터베이스는 아래 표 3-5와 같이 구성되어 있다. K-NBTC는 총 1,000명에 대해 500명씩 2개의 Set를 구성하여, 조명·표정·포즈·액세서리에 대한 인식용 DB를 구성해 성능시험에 이용된다.

» 표 3-5. 얼굴인식 성능시험용 DB 구성정보

<table>
<tr><th colspan="2">구 분</th><th colspan="2">1 set</th><th colspan="2">2 set</th></tr>
<tr><td colspan="2">인 원 (명)</td><td colspan="2">500명</td><td colspan="2">500명</td></tr>
<tr><td colspan="2">등록용DB</td><td colspan="2">500장</td><td colspan="2">500장</td></tr>
<tr><td rowspan="5">인식용 DB</td><td></td><td>분류</td><td>DB 수</td><td>분류</td><td>DB 수</td></tr>
<tr><td>1</td><td>조명방향(8)</td><td>4,000장
(500*8)</td><td>조명방향(8)</td><td>4,000장
(500*8)</td></tr>
<tr><td>2</td><td>표정변화(4)</td><td>2,000장
(500*4)</td><td>표정변화(4)</td><td>2,000장
(500*4)</td></tr>
<tr><td>3</td><td>포즈(3)</td><td>1,500장
(500*3)</td><td>포즈(3)</td><td>1,500장
(500*3)</td></tr>
<tr><td>4</td><td>액세서리(2)</td><td>1,000장
(500*2)</td><td>액세서리(2)</td><td>1,000장
(500*2)</td></tr>
<tr><td rowspan="2">총영상</td><td colspan="2">9,000장</td><td colspan="2">9,000장</td></tr>
<tr><td colspan="4">18,000장</td></tr>
<tr><td>촬영조건</td><td colspan="4">– 등록용DB : 정면/주백색(형광등)/무표정/정면조명/정면포즈
– 조명방향(8종류): 0°, 45°, 90°, 135°, 180°, 225°, 270°, 315°
※ 0°(정면, 주광색/백열등), 45°~315°(주백색/형광등)
– 표정(4종류): 웃는 표정, 화난 표정, 눈감은 표정, 놀란 표정
– 포즈(3종류): 22.5°(Left, Right), 45°(Left)
– 액세서리(2종류): 헤어밴드, 안경</td></tr>
<tr><td>형식</td><td colspan="4">– 이미지크기(가로x세로) : 640×480
– 파일 포맷 : JPG 파일</td></tr>
</table>

(4) DLL 구현 예제 설명

- 본 절에서는 DLL 개발을 위한 구현 예제를 설명하며, 예제에 사용되는 파일은 홈페이지(www. ksecurity/bio)의 자료실에서 다운로드할 수 있다.

- 다운로드 한 IndustryAlgorithm(얼굴) 폴더에서
  VC++ 6 사용자는 프로젝트 파일 IndustryAlgorithm_vc6.dsw,
  VC++ 2005 사용자는 프로젝트 파일 IndustryAlgorithm_vc2005.sln,
  VC++ 2010 사용자는 프로젝트 파일 IndustryAlgorithm_vc2010.sln을 선택하고, 이어서 관련 파일 중 BioApi.cpp를 선택한다.
  핵심 DLL 인터페이스 함수가 구현되어 있으며, 별표 3 DLL 프로그램 예시(BioApi.cpp)를 참고하면 된다.

(5) 주의사항

- 신청인이 제출한 DLL 파일 이외에 부속 파일들(dll, exe)을 포함할 경우, 성능평가 도구 내에서 DLL이 로드되어 실행될 때 부속 파일들을 로드하지 못하고, 실행이 되지 않는 문제가 발생할 수 있다.

- 이는 시험도구 프로그램의 위치가 피평가 모듈 파일들의 위치와 다른 경로상에 존재하기 때문이다. 이를 해결하기 위해서는 기본적으로 피평가 모듈 내에서 부속 파일들을 접근할 때 현재 실행되는 프로그램(시험도구)의 위치에서 찾지 말고, 피평가 모듈의 위치에서 찾도록 구현되어야 한다. 이를 위해 아래를 구현해야 한다.

- 부속 DLL이나 별도의 부속 파일을 이용하는 경우, 부속 파일들은 DLL과 동일한 경로상에 위치해야 하며, DLL의 경로를 얻는 방법은 별표 3 DLL 프로그램 예시(BioApi.cpp)를 참고하면 된다.

4. 정맥인식 성능시험

(1) 개요

- 정맥인식 성능시험을 진행하기 위해서는 아래 절에서 설명하는 윈도우용 DLL을 구현해서 제출하여야 한다. K-NBTC 담당자는 신청인의 DLL로 정맥인식 성능시험을 하게 되며, 시험결과가 인증기준을 통과하면, 진흥원에서는 인증서를 발급한다.

(2) DLL 개발 방법

- 신청인이 개발할 DLL은 다음과 같은 입출력을 가지는 3가지 함수를 반드시 내장하여야 한다. 이 3가지 함수가 성능시험 도구의 인터페이스가 된다.

(1) KISA_Enroll

- KISA_Enroll은 정맥 이미지를 읽어서 추출한 특징량을 저장하는 함수이며, 특히 등록용 이미지를 읽고, 이로부터 특징량을 추출/저장하는 함수이다. 이를 줄여서 “등록용 특징량 추출 함수”라고 한다.

  – 등록용 특징량 추출함수

```
KISADLL bool KISA_Enroll(
        BYTE *VeinIMG,          // IN: 등록용 정맥 이미지 픽셀 주소
        int width,              // IN: 이미지 너비
        int height,             // IN: 이미지 높이
        BYTE *EnrollVeinCode    // OUT: 추출된 특징량이 저장될 공간
                                // 시험도구에서는 최대 5000byte까지 허용함
```

  리턴 값은 성공 시 TRUE 또는 1, 실패 시 FALSE 또는 0을 리턴한다.

- KISA_Enroll 함수의 변수에 대한 설명은 아래와 같다.
  - (VeinIMG) 정맥 이미지의 픽셀 주소이다. (VeinIMG+0)은 손등정맥 또는 지정맥, 장정맥 이미지의 좌상단 첫 번째 픽셀 주소를 가리킨다.
    또한, 이미지의 한 픽셀은 1 byte 메모리 크기를 갖는다.
    손등정맥 이미지 크기는 640(W)x480(H)이므로, 289,800bytes 크기이며, 지정맥 이미지 크기는 1,600(W)x1,200(H)이므로, 1,920,000bytes 이고, 장정맥 이미지 크기는 512(W)x512(H) 262,144bytes 이다.
  - (width) 정맥 이미지의 너비이다. 손등정맥은 640이며, 지정맥은 1,600이고, 장정맥은 512이다.
  - (height) 정맥 이미지의 높이이다. 손등정맥은 480이며, 지정맥은 1,200이고, 장정맥은 512이다.
  - (EnrollVeinCode) 신청인의 알고리즘이 정맥 이미지에서 특징량을 추출한 후, 저장하는 특징량 데이터의 메모리 주소값을 의미한다. 이때 메모리 크기는 5,000바이트를 넘어서는 안된다.

(2) KISA_Attempt

- KISA_Attempt는 정맥 이미지를 읽어서 추출한 특징량을 저장하는 함수이며, 특히 인식용 이미지를 읽고, 이로부터 특징량을 추출/저장하는 함수이다. 이를 줄여서 “인식용 특징량 추출 함수”라고 한다.

- 인식용 특징량 추출 함수인 KISA_Attempt는 상기 KISA_Enroll과 함수의 기능 및 변수의 의미가 완전히 동일하다.
  - 인식용 특징량 추출함수

```
KISADLL bool KISA_Attempt(
        BYTE *VeinIMG,          // IN: 인식용 영상 이미지 픽셀 주소
        int width,              // IN: 이미지 너비
        int height,             // IN: 이미지 높이
        BYTE *RecogVeinCode     // OUT: 추출된 특징량이 저장될 공간
                                // 시험도구에서는 최대 5000byte까지 허용함
        )
```

(3) KISA_Compare

- KISA_Compare는 KISA_Enroll의 EnrollVeinCode와 KISA_Attempt의 RecogVein Code에 각각 저장된 정맥 이미지의 특징량을 비교하여, 정합점수를 도출하는 함수이다.

  – 비교용 함수

```
KISADLL float KISA_Compare(
        BYTE *EnrollVeinCode,       // IN: KISA_Enroll 함수에서
                                    // OUT파라미터로 넘겨줬던 특징량
        BYTE *RecogVeinCode         // IN: KISA_Attempt 함수에서
                                    // OUT파라미터로 넘겨줬던 특징량
        )
```

  리턴 값은 비교결과 나온 정합점수를 리턴하면 된다. 0(동일 정맥)과 1(타인 정맥) 사이의 값이어야 하며, 최대 소수점 3번째 자리까지 그 값을 나타낼 수 있어야 한다.

(3) 성능시험용 데이터베이스

- 정맥인식 성능시험용 DB는 사전시험용 성능 DB 구성정보와 동일하나, DB 숫자가 더 크다.
- 손등정맥인식 성능시험용 DB는 총 20,400장이며, 손을 고정한 상태에서 촬영한 A 타입과 손을 움직이는 상태에서 촬영한 B타입 2종으로 구성되어 있다.
- 지정맥인식 성능시험용 DB는 총 40,800장이며, 손가락을 고정한 상태에서 촬영한 A 타입과 손가락을 움직이는 상태에서 촬영한 B타입 2종으로 구성되어 있다.
- 장정맥인식 성능시험용 DB는 총 24,000장이며, 손바닥 위치 변화와 정맥 패턴의 선명도 차이를 포함한 다양한 조건은 반영하고 있다. 다른 정맥과 달리 별도 타입을 구분하지 않는다.

표 3-6. 손등정맥인식 성능시험용 DB 구성정보

구 분	내 용	
인원(명)	510명	
대상 손	양 손(오른손, 왼손)	
영상타입	A타입(고정형)	B타입(움직임형)
	손을 장치에 고정한 상태에서 촬영	손을 장치에 넣었다 뺐다를 반복하여 촬영
총 영상(장)	510명×양 손×10장 = 10,200장	510명×양 손×10장 = 10,200장
	총 20,400장	
가로 x 세로(Pixel)	640x480	
파일포맷	BMP 파일	

표 3-7. 지정맥인식 성능시험용 DB 구성정보

구 분	내 용	
인원(명)	510명	
대상 손	양 손(오른손, 왼손) 검지, 중지	
영상타입	A타입(고정형)	B타입(움직임형)
	손가락을 장치에 고정한 상태에서 촬영	손가락을 장치에 넣었다 뺐다를 반복하여 촬영
총 영상(장)	510명×(양 손 검지, 중지) ×10장 = 20,400장	510명×(양 손 검지, 중지) ×10장 = 20,400장
	총 40,800장	
가로 x 세로(Pixel)	1,600x1,200	
파일포맷	BMP 파일	

표 3-8. 장정맥인식 성능시험용 DB 구성정보

구 분	내 용
인원(명)	400명
대상 손	양 손(오른손, 왼손)
총 영상(장)	400명×(양 손)×30장 ※ 동일 손바닥의 위치 변화 및 압력에 따른 정맥 선명도 차이
	총 24,000장
가로 x 세로(Pixel)	512x512
파일포맷	BMP 파일

※ 장정맥 이미지는 AI 기술을 기반으로 제작되었으며, 시험 적합성 검증이 완료된 이미지만 활용

(4) DLL 구현 예제 설명

- 본 절에서는 DLL 개발을 위한 구현 예제를 설명하며, 예제에 사용되는 파일은 홈페이지(www. ksecurity/bio)의 자료실에서 다운로드할 수 있다.

- 다운로드 한 IndustryAlgorithm(정맥) 폴더에서
  VC++ 6 사용자는 프로젝트 파일 IndustryAlgorithm_vc6.dsw,
  VC++ 2005 사용자는 프로젝트 파일 IndustryAlgorithm_vc2005.sln,
  VC++ 2010 사용자는 프로젝트 파일 IndustryAlgorithm_vc2010.sln을 선택하고, 이어서 관련 파일 중 BioApi.cpp를 선택한다.
  핵심 DLL 인터페이스 함수가 구현되어 있으며, 별표 3 DLL 프로그램 예시(BioApi.cpp)를 참고하면 된다.

(5) 주의사항

- 신청인이 제출한 DLL 파일 이외에 부속 파일들(dll, exe)을 포함할 경우, 시험도구 내에서 DLL이 로드되어 실행될 때 부속 파일들을 로드하지 못하고, 실행이 되지 않는 문제가 발생할 수 있다.

- 이는 시험도구 프로그램의 위치가 피평가 모듈 파일들의 위치와 다른 경로상에 존재하기 때문이다. 이를 해결하기 위해서는 기본적으로 피평가 모듈 내에서 부속 파일들을 접근할 때 현재 실행되는 프로그램(시험도구)의 위치에서 찾지 말고, 피평가 모듈의 위치에서 찾도록 구현되어야 한다. 이를 위해 아래를 구현해야 한다.

- 부속 DLL이나 별도의 부속 파일을 이용하는 경우, 부속 파일들은 DLL과 동일한 경로상에 위치해야 하며, DLL의 경로를 얻는 방법은 별표 3 DLL 프로그램 예시(BioApi.cpp)를 참고하면 된다.

5. 홍채인식 성능시험

(1) 개요

- 홍채인식 성능시험을 진행하기 위해서는 아래 절에서 설명하는 윈도우용 DLL을 구현해서 제출하여야 한다. K-NBTC 담당자는 신청인의 DLL로 홍채인식 성능시험을 하게 되며, 시험결과가 인증기준을 통과하면, 진흥원에서는 인증서를 발급한다.

(2) DLL 개발 방법

- 신청인이 개발할 DLL은 다음과 같은 입출력을 가지는 3가지 함수를 반드시 내장하여야 한다. 이 3가지 함수가 성능시험도구의 인터페이스가 된다.

(1) KISA_Enroll

- KISA_Enroll은 홍채 이미지를 읽어서 추출한 특징량을 저장하는 함수이며, 특히 등록용 이미지를 읽고, 이로부터 특징량을 추출/저장하는 함수이다. 이를 줄여서 “등록용 특징량 추출 함수”라고 한다.

  – 등록용 특징량 추출함수

```
KISADLL bool KISA_Enroll(
        BYTE * img,                 // IN: 등록용 영상 이미지 픽셀 주소
        int width,                  // IN: 이미지 너비
        int height,                 // IN:이미지 높이
        BYTE * pEnroll              // OUT: 추출된 특징량이 저장될 공간
                                    // 시험도구에서는 최대 5000byte까지 허용함
        )
```

  리턴 값은 성공 시 TRUE, 실패 시 FALSE를 리턴한다.

- KISA_Enroll 함수의 변수에 대한 설명은 아래와 같다.
  - (img) 홍채 이미지의 픽셀 주소이다. (VeinIMG+0)은 홍채 이미지의 좌상단 첫 번째 픽셀 주소를 가리킨다.
    또한, 이미지의 한 픽셀은 1 byte 메모리 크기를 갖는다.
    홍채 이미지 크기는 630(W)x460(H)이므로, 289,800 bytes 크기이다.
  - (width) 홍채 이미지의 너비이다.
  - (height) 홍채 이미지의 높이이다.
  - (pEnroll) 신청인의 알고리즘이 홍채 이미지에서 특징량을 추출한 후, 저장하는 특징량 데이터의 메모리 주소값을 의미한다. 이때 메모리 크기는 5,000바이트를 넘어서는 안 된다.

(2) KISA_Attempt

- KISA_Attempt는 홍채 이미지를 읽어서 추출한 특징량을 저장하는 함수이며, 특히 인식용 이미지를 읽고, 이로부터 특징량을 추출/저장하는 함수이다. 이를 줄여서 “인식용 특징량 추출 함수”라고 한다.

- 인식용 특징량 추출 함수인 KISA_Attempt는 상기 KISA_Enroll과 함수의 기능 및 변수의 의미가 완전히 동일하다.
  - 인식용 특징량 추출함수

```
KISADLL bool KISA_Attempt(
        BYTE *img,                  // IN: 비교용 영상 이미지 주소
        int width,                  // IN: 이미지 너비
        int height,                 // IN: 이미지 높이
        BYTE * pAttempt             // OUT: 추출된 특징량이 저장될 공간
                                    // 시험도구에서는 최대 5000byte까지 허용함
        )
```

리턴 값은 성공 시 1, 실패 시 0을 리턴한다.

(3) KISA_Compare

- KISA_Compare는 KISA_Enroll의 pEnroll과 KISA_Attempt의 pAttempt에 각각 저장된 홍채 이미지의 특징량을 비교하여, 정합점수를 도출하는 함수이다.

  – 비교용 함수

```
KISADLL float KISA_Compare(
        BYTE * pEnroll,              // IN: KISA_Enroll 함수에서
                                     // OUT파라미터로 넘겨줬던 특징량
        BYTE * pAttempt              // IN: KISA_Attempt 함수에서
                                     // OUT파라미터로 넘겨줬던 특징량
        )
```

  리턴 값은 비교결과 나온 정합점수를 리턴하면 된다. 0(동일홍채)과 1(타인홍채) 사이의 값이어야 하며, 최대 소수점 3째 자리까지 그 값을 나타낼 수 있어야 한다.

(3) 성능시험용 데이터베이스

- 홍채인식 성능시험용 DB는 사전시험용 성능 DB 구성정보와 동일하나, DB 크기가 더 크다.

표 3-9. 홍채인식 성능시험용 DB 구성정보

구 분	내 용	
인원(명)	510명	
대상 눈	양쪽 눈(오른쪽 눈, 왼쪽 눈) ※ 안경착용 영상 미포함, 콘택트렌즈 착용 영상포함	
영상타입	A타입(정면 응시)	B타입(비스듬히 응시)
	홍채를 장치에 정면으로 응시한 상태에서 촬영	홍채를 장치에 5-10도 off-angle에서 촬영
총 영상(장)	510명×양쪽 눈×10장 = 10,200장	510명×양쪽 눈×10장 = 10,200장
	총 20,400장	
가로 x 세로(Pixel)	630x460	
파일포맷	BMP 파일	

(4) DLL 구현 예제 설명

- 본 절에서는 DLL 개발을 위한 구현 예제를 설명하며, 예제에 사용되는 파일은 홈페이지(www. ksecurity/bio)의 자료실에서 다운로드할 수 있다.

- 다운로드 한 IndustryAlgorithm(홍채) 폴더에서
  VC++ 6 사용자는 프로젝트 파일 IndustryAlgorithm_vc6.dsw,
  VC++ 2005 사용자는 프로젝트 파일 IndustryAlgorithm_vc2005.sln,
  VC++ 2010 사용자는 프로젝트 파일 IndustryAlgorithm_vc2010.sln을 선택하고, 이어서 관련 파일 중 BioApi.cpp를 선택한다.
  핵심 DLL 인터페이스 함수가 구현되어 있으며, 별표 3 DLL 프로그램 예시(BioApi.cpp)를 참고하면 된다.

(5) 주의사항

- 신청인이 제출한 DLL 파일 이외에 부속 파일들(dll, exe)을 포함할 경우, 시험도구 내에서 DLL이 로드되어 실행될 때 부속 파일들을 로드하지 못하고, 실행이 되지 않는 문제가 발생할 수 있다.

- 이는 시험도구 프로그램의 위치가 피평가 모듈 파일들의 위치와 다른 경로상에 존재하기 때문이다. 이를 해결하기 위해서는 기본적으로 피평가 모듈 내에서 부속 파일들을 접근할 때 현재 실행되는 프로그램(시험도구)의 위치에서 찾지 말고, 피평가 모듈의 위치에서 찾도록 구현되어야 한다. 이를 위해 아래를 구현해야 한다.

- 부속 DLL이나 별도의 부속 파일을 이용하는 경우, 부속 파일들은 DLL과 동일한 경로상에 위치해야 하며, DLL의 경로를 얻는 방법은 별표 3 DLL 프로그램 예시(BioApi.cpp)를 참고하면 된다.

[별표 1] 시나리오 종류에 따른 구현 함수

3.1 시나리오① : Verification BSP

– 필수 구현 함수

BioSPI_BSPLoad(), BioSPI_BSPUnload(), BioSPI_BSPAttach(), BioSPI_BSPDetach(), BioSPI_QueryUnits(), BioSPI_FreeBIRHandle(), BioSPI_GetBIRFromHandle(), BioSPI_GetHeaderFromHandle(), BioSPI_EnableEvents(), BioSPI_Enroll(), BioSPI_Verify(), BioSPI_Cancel(), BioSPI_Free()

3.2 시나리오② : Identification BSP

– 필수 구현 함수

BioSPI_BSPLoad(), BioSPI_BSPUnload(), BioSPI_BSPAttach(), BioSPI_BSPDetach(), BioSPI_QueryUnits(), BioSPI_FreeBIRHandle(), BioSPI_GetBIRFromHandle(), BioSPI_GetHeaderFromHandle(), BioSPI_EnableEvents(), BioSPI_Enroll(), BioSPI_Verify(), BioSPI_Identify(), BioSPI_Cancel(), BioSPI_Free()

3.3 시나리오③ : Capture BSP

– 필수 구현 함수

BioSPI_BSPLoad(), BioSPI_BSPUnload(), BioSPI_BSPAttach(), BioSPI_BSPDetach(), BioSPI_QueryUnits(), BioSPI_FreeBIRHandle(), BioSPI_GetBIRFromHandle(), BioSPI_GetHeaderFromHandle(), BioSPI_EnableEvents(), BioSPI_Capture(), BioSPI_Cancel(), BioSPI_Free()

3.4 시나리오④ : Verification Engine

– 필수 구현 함수

BioSPI_BSPLoad(), BioSPI_BSPUnload(), BioSPI_BSPAttach(),

BioSPI_BSPDetach(), BioSPI_FreeBIRHandle(),
BioSPI_GetBIRFromHandle(),
BioSPI_GetHeaderFromHandle(), BioSPI_EnableEvents(),
BioSPI_CreateTemplate(), BioSPI_Process(), BioSPI_VerifyMatch(),
BioSPI_Cancel(), BioSPI_Free()

3.5 시나리오⑤ : Identification Engine

– 필수 구현 함수

BioSPI_BSPLoad(), BioSPI_BSPUnload(), BioSPI_BSPAttach(),
BioSPI_BSPDetach(), BioSPI_FreeBIRHandle(),
BioSPI_GetBIRFromHandle(),
BioSPI_GetHeaderFromHandle(), BioSPI_EnableEvents(),
BioSPI_CreateTemplate(), BioSPI_Process(), BioSPI_VerifyMatch(),
BioSPI_IdentifyMatch(), BioSPI_Cancel(), BioSPI_Free()

3.6 시험 시나리오에 따른 필수 함수

함수 \ 시나리오	① Verification BSP	② Identification BSP	③ Capture BSP	④ Verification Engine	⑤ Identification Engine
Component Management Functions					
BioSPI_BSPLoad	○	○	○	○	○
BioSPI_BSPUnload	○	○	○	○	○
BioSPI_BSPAttach	○	○	○	○	○
BioSPI_BSPDetach	○	○	○	○	○
BioSPI_QueryUnits	○	○	○	–	–
Handle Functions					
BioSPI_FreeBIRHandle	○	○	○	○	○
BioSPI_GetBIRFromHandle	○	○	○	○	○
BioSPI_GetHeaderFromHandle	○	○	○	○	○

함수 \ 시나리오	① Verification BSP	② Identification BSP	③ Capture BSP	④ Verification Engine	⑤ Identification Engine
Callback and Event Functions					
BioSPI_EnableEvents	○	○	○	○	○
Biometric Functions					
BioSPI_Capture	–	–	○	–	–
BioSPI_CreateTemplate	–	–	–	○	○
BioSPI_Process	–	–	–	○	○
BioSPI_VerifyMatch	–	–	–	○	○
BioSPI_IdentifyMatch	–	–	–	–	○
BioSPI_Enroll	○	○	–	–	–
BioSPI_Verify	○	○	–	–	–
BioSPI_Identify	–	○	–	–	–
Utility Functions					
BioSPI_Cancel	○	○	○	○	○
BioSPI_Free	○	○	○	○	○
필수 함수 총계	13개	14개	12개	13개	14개

함수 \ 시나리오	① Verification BSP	② Identification BSP	③ Capture BSP	④ Verification Engine	⑤ Identification Engine
Component Management Functions					
BioSPI_BSPLoad	○	○	○	○	○
BioSPI_BSPUnload	○	○	○	○	○
BioSPI_BSPAttach	○	○	○	○	○
BioSPI_BSPDetach	○	○	○	○	○
BioSPI_QueryUnits	○	○	○	–	–
Handle Functions					
BioSPI_FreeBIRHandle	○	○	○	○	○

함수 \ 시나리오	① Verification BSP	② Identification BSP	③ Capture BSP	④ Verification Engine	⑤ Identification Engine
BioSPI_GetBIRFromHandle	○	○	○	○	○
BioSPI_GetHeaderFromHandle	○	○	○	○	○
Callback and Event Functions					
BioSPI_EnableEvents	○	○	○	○	○
Biometric Functions					
BioSPI_Capture	–	–	○	–	–
BioSPI_CreateTemplate	–	–	–	○	○
BioSPI_Process	–	–	–	○	○
BioSPI_VerifyMatch	–	–	–	○	○
BioSPI_IdentifyMatch	–	–	–	–	○
BioSPI_Enroll	○	○	–	–	–
BioSPI_Verify	○	○	–	–	–
BioSPI_Identify	–	○	–	–	–
Utility Functions					
BioSPI_Cancel	○	○	○	○	○
BioSPI_Free	○	○	○	○	○
필수 함수 총계	13개	14개	12개	13개	14개

[별표 2] 테스트어서션 및 예시

```
<?xml version="1.0"?>
<package name="01f6c6f0-0c19-1085-97fe-0002a5d5fd2e">
        <author>
                ISO/IEC JTC1 SC37
        </author>
        <description>
                ...
        </description>
        <assertion name="BioSPI_BSPLoad_ValidParam" model="BSPTesting">
                <description>
                        ...
                </description>
                <input name="_BspUuid"/>
                <input name="_BioAPINotifyCallback"/>
                <input name="_BFPEnumerationHandler"/>
                <input name="_MemoryFreeHandler"/>
                <invoke activity="BioSPI_BspLoad">
                        <input name="BspUuid" var="_BspUuid"/>
                        <input name="BioAPIdNotifyCallback" var="_BioAPINotifyCallback"/>
                        <input                                  name="BFPEnumerationHandler"
var="_BFPEnumerationHandler"/>
                        <input name="MemoryFreeHandler" var="_MemoryFreeHandler"/>
                </invoke>
        </assertion>
        <activity name="BioSPI_BspLoad">
                <input name="BspUuid"/>
                <input name="BioAPIdNotifyCallback"/>
                <input name="BFPEnumerationHandler"/>
                <input name="MemoryFreeHandler"/>
                <invoke function="BioSPI_BSPLoad">
                        <input name="BSPUuid" var="BspUuid"/>
                        <input name="BioAPINotifyCallback" var="BioAPIdNotifyCallback"/>
                        <input                                  name="BFPEnumerationHandler"
var="BFPEnumerationHandler"/>
                        <input name="MemoryFreeHandler" var="MemoryFreeHandler"/>
                        <return setvar="return"/>
                </invoke>
                <assert_condition>
                        <description>
                                The function BioSPI_BSPLoad has returned BioAPI_OK.
                        </description>
                        <equal_to var1="return" var2="__BioAPI_OK"/>
                </assert_condition>
                <invoke function="BioSPI_BSPUnload">
                        <input name="BSPUuid" var="BspUuid"/>
                <return setvar="return"/>      </invoke>
                <assert_condition response_if_false="undecided">
                        <equal_to var1="return" var2="__BioAPI_OK"/>
                </assert_condition>
        </activity>
</package>
```

테스트어서션을 실행하기 위해서는 테스트어서션의 입력 파라미터 값이 지정되어야 한다.
위 예시에서는

<input name="_BspUuid"/>, <input name="_BioAPINotifyCallback"/>,

<input name="_BFPEnumerationHandler"/>, <input name="_MemoryFreeHandler"/>

등과 같이 4개의 입력 파라미터가 필요하다. 이 입력 값에 따라 테스트어서션 실행 결과가 달라질 수 있기 때문에 입력 파라미터 결정은 중요하다.

ID	어서션 명칭	목적
1a	BioSPI_BSPLoad_InvalidUUID	Verify if calling the function BioSPI_BSPLoad with an invalid input parameter UUID returns BioAPIERR_H_FRAMEWORK_IN VALID_UUID.
1b	BioSPI_BSPLoad_ValidParam	Verify if calling BioSPI_BSPLoad with valid input parameters returns BioAPI_OK.
2a	BioSPI_BSPUnload_ValidParam	Verify if calling BioSPI_BSPUnload with valid input parameters returns BioAPI_OK.
2b	BioSPI_BSPUnload_InvalidUUID	Verify if calling BioSPI_BSPUnload with an invalid input parameter UUID returns BioAPIERR_INVALID_UUID.
2c	BioSPI_BSPUnload_UnmatchedLoad	Verify if calling BioSPI_BSPUnload without a matching call to BioSPI_BSPLoad returns BioAPIERR_BSP_NOT_LOADED.
2d	BioSPI_BSPUnload_Confirm	Verify if calling BioSPI_BSPUnload truly unloads the BSP.
3a	BioSPI_BSPAttach_ValidParam	Verify if a call to the function BioSPI_BSPAttach with valid input parameters returns BioAPI_OK.
3b	BioSPI_BSPAttach_InvalidUUID	Verify if a call to the function BioSPI_BSPAttach returns BioAPIERR_INVALID_UUID when the input UUID is invalid.
3c	BioSPI_BSPAttach_InvalidVersion	Verify if a call to the function BioSPI_BSPAttach returns BioAPIERR_INCOMPATIBLE_VERSION when the caller specifies an incompatible version.
3d	BioSPI_BSPAttach_InvalidBSPHandle	Verify if the function BioSPI_BSPAttach returns an error

ID	어서션 명칭	목적
		when called with an invalid BSP handle
4a	BioSPI_BSPDetach_ValidParam	Verify if a call to the function BioSPI_BSPDetach with a valid module handle returns BioAPI_OK.
4b	BioSPI_BSPDetach_InvalidBSPHandle	Verify if a call to BioSPI_BSPDetach with an invalid module handle returns an error.
4c	BioSPI_BSPDetach_Confirm	Verify if a call to the function BioSPI_BSPDetach truly terminates the attach session.
5a	BioSPI_FreeBIRHandle_ValidParam	Verify if calling the function BioSPI_FreeBIRHandle with a valid BIR handle frees the BIR handle.
5b	BioSPI_FreeBIRHandle_InvalidBSPHandle	Verify if calling BioSPI_FreeBIRHandle with an invalid BSP handle returns an error.
5c	BioSPI_FreeBIRHandle_InvalidBIRHandle	Verify if calling BioSPI_FreeBIRHandle with an invalid BIR handle returns an error.
6a	BioSPI_GetBIRFromHandle_ValidParam	Verify if calling BioSPI_GetBIRFromHandle with valid parameters returns BioAPI_OK.
6b	BioSPI_GetBIRFromHandle_InvalidBSPHandle	Verify if calling BioSPI_GetBIRFromHandle with an invalid BSP handle returns an error.
6c	BioSPI_GetBIRFromHandle_InvalidBIRHandle	Verify if calling the function BioSPI_GetBIRFromHandle with an invalid BIR handle returns an error.
7a	BioSPI_GetHeaderFromHandle_ValidParam	Verify if a call to the function BioSPI_GetHeaderFromHandle with valid input parameters returns BioAPI_OK.
7b	BioSPI_GetHeaderFromHandle_InvalidBSPHandle	Verify if invoking the function BioSPI_GetHeaderFromHandle with an invalid module handle returns an error.
7c	BioSPI_GetHeaderFromHandle_InvalidBIRHand	Verify if a call to the function BioSPI_GetHeaderFromHandle with an invalid BIR handle returns an error.
7d	BioSPI_GetHeaderFromHandle_BIRHandleNotFreed	Verify that after a call to the function BioSPI_GetHeaderFromHandle, the BIR handle has not been freed.
8a	BioSPI_EnableEvents_ValidParam	Verify BioSPI_EnableEvents with valid input parameters.
8b	BioSPI_EnableEvents_InvalidBSPHandle	Verify BioSPI_EnableEvents with an invalid module handle.
9a	BioSPI_Capture_AuditData	Verify the function BioSPI_Capture with AuditData having

ID	어서션 명칭	목적
		a non-NULL value.
9b	BioSPI_Capture_ReturnQuality	Verify if the header of the captured BIR contains a valid quality value (in the range 0-100).
9c	BioSPI_Capture_IntermediateProcessedBIR	Verify if the BIR returned by the function BioSPI_Capture has a processed level of either INTERMEDIATE or PROCESSED.
9d	BioSPI_Capture_InvalidBSPHandle	Verify if calling BioSPI_Capture with an invalid BSP handle returns an error BioAPIERR_INVALID_BSP_HANDLE.
10a	BioSPI_CreateTemplate_Payload Supported	Verify BioSPI_CreateTemplate with valid parameters and payload.
10b	BioSPI_CreateTemplate_BIRHeaderQuality	Verify the function BioSPI_CreateTemplate with valid parameters and the returned quality value.
10c	BioSPI_CreateTemplate_OutputBIRDataType	Verify BioSPI_CreateTemplate with valid parameters. The new template BIR is expected to have the processed level PROCESSED.
10d	BioSPI_CreateTemplate_OutputBIRPurpose	Verify if the purpose of the created template is the same as the purpose of the captured BIR.
10e	BioSPI_CreateTemplate_InputBIRDataType	Verify BioSPI_CreateTemplate with an input BIR that has an invalid processed level.
10f	BioSPI_CreateTemplate_Inconsistent_Purpose	Verify function BioSPI_CreateTemplate with an invalid input BIR purpose.
11a	BioSPI_Process_ValidParam	Verify BioSPI_Process with valid parameters and the returned processed level.
11b	BioSPI_Process_BIRHeaderQuality	Verify BioSPI_Process with valid parameters and if the returned quality is valid.
11c	BioSPI_Process_OutputBIRPurpose	Verify BioSPI_Process with valid parameters and if the purpose of the processed BIR is the same as the purpose of the captured BIR.
11d	BioSPI_Process_BuildsProcessedBIR	Verify BioSPI_Process with valid parameters and the returned processed level.
11e	BioSPI_Process_InputBIRDataType	Verify BioSPI_Process with an input BIR having a processed level of PROCESSED and if the BioSPI_Process call fails.

ID	어서션 명칭	목적
12a	BioSPI_VerifyMatch_ValidParam	Verify if calling BioSPI_VerifyMatch with valid input parameters returns BioAPI_OK.
12b	BioSPI_VerifyMatch_Payload	Verify the support of payload in the function BioSPI_VerifyMatch. The function is expected to return BioAPI_OK.
12c	BioSPI_VerifyMatch_Inconsistent_Purpose	Verify BioSPI_VerifyMatch with a BIR whose purpose is invalid for the function. The function is expected to return BioAPIERR_BSP_INCONSISTENT_PURPOSE.
13a	BioSPI_Enroll_ValidParam	Verify if calling BioSPI_Enroll with valid input parameters returns BioAPI_OK.
13b	BioSPI_Enroll_Payload	Verify if calling BioSPI_Enroll with a payload returns BioAPI_OK.
13c	BioSPI_Enroll_AuditData	Verify BioSPI_Enroll with a non-NULL AuditData parameter. The function is expected to return audit data if the BSP supports audit data.
13d	BioSPI_Enroll_BIRHeaderQuality	Verify if calling the function BioSPI_Enroll returns a valid quality value in the new template BIR's header.
14a	BioSPI_Verify_ValidParam	Verify if calling the function BioSPI_Verify with valid input parameters returns BioAPI_OK.
14b	BioSPI_Verify_Payload	Verify if calling the function BioSPI_Verify with a payload returns BioAPI_OK.
14c	BioSPI_Verify_AuditData	Verify if calling BioSPI_Verify with audit data returns BioAPI_OK.

[별표 3] DLL 프로그램 예시(BioApi.cpp)

– 지문 성능시험용 DLL 프로그램 예제(BioApi.cpp)

```
#define KISADLL          extern "C" __declspec(dllexport)
KISADLL int KISA_Enroll(
IN int iType,                          // pImage의 센서 종류 (표 3-3 참조)
IN unsigned char* pImage,              // 센서 종류에 따른 등록용 지문 이미지 픽셀 주소
OUT unsigned char* pEnroll,            // 추출된 특징량이 저장되는 메모리 주소
OUT int* piEnrollSize,                 // 추출된 특징량 크기 (1,000,000byte 보다 작아야
함)
OUT int* piErrorCode,                  // 신청인이 지정하는 에러 코드
OUT char arErrorMessage[256]           // 신청인이 지정하는 에러 메시지
{
        // 센서 종류에 따른 분기
        switch(iType)
        {
        case 5:   // 센서타입 DIG00
                  // pImage에 좌상단 이미지 포인터가 담긴다.
        break;
        case 7:   // 센서타입 PER00
        break;
        case 8:   // 센서타입 NIT00
        break;
        case 9:   // 센서타입 HUN00
        break;
        }
// pImage를 읽어서 지역변수 Enroll에 1,000byte 크기의 특징량을 문자열인
//"5518bfe3fec4dbce392ddd4482812f7b"로 추출했다고 가정한다
        char Enroll[1000] ="5518bfe3fec4dbce392ddd4482812f7b";

// 추출된 특징량을 OUT 파라미터로 지정된 pEnroll에 memcpy로 넣어준다.
        memcpy(pEnroll, Enroll, 1000);

// 특징량의 크기 1,000byte를 OUT 파라미터로 지정된 piEnrollSize에 넣어준다.
        *piEnrollSize = 1000;

        // 신청인이 지정하는 에러코드 (여기서는 예시로서 1만 정의)
        *piErrorCode = 1;
```

```
	// 신청인이 지정하는 에러메시지 (256byte 안의 char 문자열을 출력할 수 있음)
	char strMsg[256] = "1번 에러 발생";
	memcpy(arErrorMessage, strMsg, 256);

	// 문제없이 실행된 경우 1을 리턴한다.
	// 0을 리턴하면 시험도구에서는 에러로 처리하며 위에서 신청인이
	// 지정한 해당 에러코드와 에러메시지를 출력해준다.
	return 1;
}

// 인식용 지문 이미지의 주소를 받아와 특징량을 추출하는 함수
// 인식용이라는 차이만 있을 뿐 KISA_Enroll함수와 처리가 동일하다.
KISADLL int KISA_Attempt (
IN int iType,                    // pImage의 센서 종류 (표 3-3 참조)
IN unsigned char* pImage,          // 센서 종류에 따른 크기의 인식용 지문 이미지 픽셀
주소
OUT unsigned char* pAttempt,     // 추출된 특징량이 저장되는 메모리 주소
OUT int* piAttemptSize,          // 추출된 특징량 크기
OUT int* piErrorCode,            // 신청인이 지정하는 에러 코드
OUT char arErrorMessage[256]     // 신청인이 지정하는 에러 메시지
{
	switch(iType)
	{
	case 5:   // 센서타입 DIG00
	          // pImage에 좌상단 이미지 포인터가 담긴다.
	break;
	case 7:   // 센서타입 PER00
	break;
	case 8:   // 센서타입 NIT00
	break;
	case 9:   // 센서타입 HUN00
	break;
	}
// pImage를 읽어서 지역변수 Enroll에 1000byte 크기의 특징량을 추출했다고 가정
	char Enroll[1000] =“b74bdafc093aeb4b78698b46b3d1e0b1”;
```

```
// 추출된 특징량을 OUT 파라미터로 지정된 pEnroll에 memcpy로 넣어준다.
        memcpy(pAttempt, Enroll, 1000);
// 특징량의 크기 1000byte를 OUT 파라미터로 지정된 piEnrollSize에 넣어준다
        *piAttemptSize = 1000;

// 신청인이 지정하는 에러코드 (여기서는 예시로서 1만 정의)
        *piErrorCode = 1;

// 신청인이 지정하는 에러메시지 ( 256byte 안의 char 문자열을 출력할 수 있음)
        char strMsg[256] ="1번 에러 발생";
        memcpy(arErrorMessage,strMsg, 256);

        // 문제없이 실행된 경우 1을 리턴한다.
        // 0을 리턴하면 시험도구에서는 에러로 처리하며 위에서 신청인이
        // 지정한 해당 에러코드와 에러메시지를 출력해준다.
        return 1;
}
// 추출된 등록용 특징량과 인식용 특징량을 비교한 결과, 정합점수를 판정하는 함수
KISADLL int KISA_Compare(
IN unsigned char* pEnroll,
// KISA_Enroll 함수에서 넘겨줬던 등록용 특징량이 저장된 메모리의 주소
IN unsigned char* pAttempt,
// KISA_Attempt 함수에서 넘겨줬던 인식용 특징량이 저장된 메모리의 주소
OUT double* pdScore,              // 0(타인 지문)에서 1(동일 지문)사이의 정합 점수
OUT int* piErrorCode,             // 신청인이 지정하는 에러 코드
OUT char arErrorMessage[256]      // 신청인이 지정하는 에러 메시지
{
        //pEnroll 과 pAttempt에 담긴 등록, 인식 영상 이미지의 특징량을 읽어와
        //비교하였다고 가정한다.

        // pdScore에 비교 결과가 되는 정합 점수를 담아준다.
        // 예를 들어 정합점수 0.821 이었을 때,
        *pdScore = 0.821;

        // 신청인이 지정하는 에러코드 (여기서는 예시로서 1만 정의)
        *piErrorCode = 1;
        // 신청인이 지정하는 에러메시지 (256byte 안의 char 문자열을 출력할 수 있음)
```

```
        char strMsg[256] = "1번 에러 발생";
        memcpy(arErrorMessage,strMsg, 256);
        // 문제없이 실행된 경우 1을 리턴한다.
        // 0을 리턴하면 시험도구에서는 에러로 처리하며 위에서 신청인이
        // 지정한 해당 에러코드와 에러메시지를 출력해준다.
        return 1;
}
```

– 얼굴인식 성능시험용 DLL 프로그램 예제(BioApi.cpp)

```
// 등록용 이미지를 데이터로 입력 받아 특징량을 리턴하는 함수
KISADLL BioAPI_RETURN  KISA_Enroll(
        BioAPI_HANDLE               ModuleHandle,
        const BioAPI_INPUT_BIR      *CapturedBIR,
        const BioAPI_INPUT_BIR      *StoredTemplate,
        BioAPI_BIR_HANDLE_PTR       NewTemplate,
        const BioAPI_DATA           *Payload)
{
        if(CapturedBIR == NULL || NewTemplate == NULL)
        {
                return !BioAPI_OK;
        }
        // 아래 코드로 pBIR→Signature→Data에 이미지 파일의 포인터가 담긴다.
        // 이것을 분석하여 특징량을 추출하도록 한다.
        BioAPI_BIR_PTR pBIR = CapturedBIR->InputBIR.BIR;

        // Enroll을 추출된 특징량이라고 가정
        char Enroll[1000] = "5518bfe3fec4dbce392ddd4482812f7b";
        // 특징량 Enroll을 담아주는 방법은 아래 두줄의 코드와 같다.
        BioAPI_HANDLE Handle = AddNewBIR(Enroll, 1000);
        *NewTemplate = Handle;
        // 담아줌
        return BioAPI_OK;
}
// 기능적으로 KISA_Attempt 함수는 KISA_Enroll 함수와 동일하다.
```

```
// 코드를 보면 실제로 KISA_Enroll을 다시 호출하는 것을 볼 수 있다.
// 따라서 KISA_Attempt는 구현 예제 코드를 그대로 사용하면 된다.
KISADLL BioAPI_RETURN  KISA_Attempt(
        BioAPI_HANDLE              ModuleHandle,
        const BioAPI_INPUT_BIR     *CapturedBIR,
        BioAPI_BIR_HANDLE_PTR   ProcessedBIR)
{
        return KISA_Enroll(NULL, CapturedBIR, NULL, ProcessedBIR, NULL);
}
// 등록용과 인식용 특징량을 비교하여 정합점수를 리턴하는 함수
KISADLL BioAPI_RETURN  KISA_Compare(
        BioAPI_HANDLE              ModuleHandle,
        const BioAPI_FAR           *MaxFARRequested,
        const BioAPI_FRR           *MaxFRRRequested,
        const BioAPI_BOOL          *FARPrecedence,
        const BioAPI_INPUT_BIR     *ProcessedBIR,
        const BioAPI_INPUT_BIR     *StoredTemplate,
        BioAPI_BIR_HANDLE          *AdaptedBIR,
        BioAPI_BOOL                *Result,
        BioAPI_FAR_PTR   FARAchieved,
        BioAPI_FRR_PTR   FRRAchieved,
        BioAPI_DATA_PTR            *Payload)
{
        // 에러가 있는 경우
        if(ProcessedBIR == NULL || StoredTemplate == NULL || Result == NULL)
        {
        return !BioAPI_OK;
        }
        // 인식용 특징량은 매개 변수인 ProcessedBIR→BIRinBSP에 담겨있다.
        // 등록용 특징량은 매개 변수인 StoredTemplate→BIRinBSP 담겨있다.
        // 둘을 비교했다 가정하면 아래처럼 정합점수를 result에 담아준다.
        *Result = 80;  // 정합점수 80
}
```

– 얼굴인식 DLL 파일의 경로를 얻기 위한 구현 예제(DllMain.cpp)

```
HMODULE g_hDLL = NULL;

BOOL APIENTRY DllMain(HANDLE _hModule, DWORD _dwReason, LPVOID _lpRes)
{
	switch(_dwReason)
	{
	case DLL_PROCESS_ATTACH:
		g_hDLL = (HMODULE)_hModule;
		break;
	case DLL_PROCESS_DETACH:
		break;
	case DLL_THREAD_ATTACH:
		break;
	case DLL_THREAD_DETACH:
		break;
	}

	return TRUE;
}

// 이 함수처럼 구현하면 dll이 속해 있는 경로를 구할 수 있음.
void GetPath()
{
	char strPath [MAX_PATH] = {0};
	::GetModuleFileName(g_hDLL, strPath, MAX_PATH);
}
```

– 정맥인식 성능시험용 DLL 프로그램 예제(BioApi.cpp)

```
// 등록용 영상 이미지의 주소를 받아와 특징량을 추출하는 함수
KISADLL bool KISA_Enroll(
	BYTE *VeinIMG,		// 등록용 영상 이미지 픽셀 주소
	int width,		// 이미지 너비
	int height,		// 이미지 높이
```

```
        BYTE *EnrollVeinCode        // OUT파라미터 추출된 특징량이 저장될 공간
                                    // 시험도구에서는 최대 5000byte까지 허용함
        )
{
        // VeinIMG로부터 영상을 읽어와 특징량을 추출한다.
        // 특징량을 아래 변수 Enroll에 담았다고 가정
        // 여기에서는 아무 값이나 대입하였음
        char Enroll[1000] = "5518bfe3fec4dbce392ddd4482812f7b";

        // 추출된 특징량을 OUT 파라미터로 지정된 EnrollVeinCode에
        // 아래와 같이 memcpy로 넣어준다.
        memcpy(EnrollVeinCode, Enroll, 1000);

        // 성공 시 true 리턴, 에러 시에는 false를 리턴한다.
        return true;
}

// 인식용 영상 이미지의 주소를 받아와 특징량을 추출하는 함수
// 인식용이라는 차이만 있을 뿐 KISA_Enroll함수와 처리가 동일
KISADLL bool KISA_Attempt(
        BYTE *VeinIMG,              // 인식용 영상 이미지 픽셀 주소
        int width,                  // 이미지 너비
        int height,                 // 이미지 높이
        BYTE *RecogVeinCode         // OUT파라미터 추출된 특징량이 저장될 공간
                                    // 시험도구에서는 최대 5000byte까지 허용함
        )
{
        // VeinIMG로부터 영상을 읽어와 특징량을 추출한다.
        // 특징량을 아래 변수 Enroll에 담았다고 가정.
        // 여기에서는 아무 값이나 대입하였음.
        char Enroll[1000] = "5518bfe3fec4dbce392ddd4482812f7b";

        // 추출된 특징량을 OUT 파라미터로 지정된 RecogVeinCode에
        // 아래와 같이 memcpy로 넣어준다.
        memcpy(RecogVeinCode, Enroll, 1000);

        // 성공 시 true 리턴, 에러 시에는 false를 리턴한다.
```

```
        return true;
}
// 추출된 등록용 특징량과 인식용 특징량을 비교한 결과, 정합점수를 판정하는 함수
KISADLL float KISA_Compare(
        BYTE *EnrollVeinCode,          // KISA_Enroll 함수에서 넘겨줬던 특징량
        BYTE *RecogVeinCode            // KISA_Attempt 함수에서 넘겨줬던 특징량
        )
{
        // EnrollVeinCode, RecogVeinCode를 비교하여 정합 점수를 리턴한다.
        float fResult;          // 정합점수를 저장하기 위한 결과 변수. 이 값을 리턴한다.

        fResult = 0.5f;         // 정합점수가 0.5일 때 ( 0부터 1사이의 값 )

        return fResult;         // 도출된 정합점수 리턴
}
```

– 정맥인식 DLL 파일의 경로를 얻기 위한 구현 예제(DllMain.cpp)

```
HMODULE g_hDLL = NULL;

BOOL APIENTRY DllMain(HANDLE _hModule, DWORD _dwReason, LPVOID _lpRes)
{
        switch(_dwReason)
        {
        case DLL_PROCESS_ATTACH:
                g_hDLL = (HMODULE)_hModule;
                break;
        case DLL_PROCESS_DETACH:
                break;
        case DLL_THREAD_ATTACH:
                break;
        case DLL_THREAD_DETACH:
                break;
        }

        return TRUE;
```

```
}

// 이 함수처럼 구현하면 dll이 속해 있는 경로를 구할 수 있음.
void GetPath()
{
	char strPath [MAX_PATH] = {0};
	::GetModuleFileName(g_hDLL, strPath, MAX_PATH);
}
```

– 홍채인식 성능시험용 DLL 프로그램 예제(BioApi.cpp)

```
// 등록용 영상 이미지의 주소를 받아와 특징량을 추출하는 함수
KISADLL bool KISA_Enroll(
	BYTE *img,			// 등록용 영상 이미지 픽셀 주소
	int width,			// 이미지 너비
	int height,			// 이미지 높이
	BYTE * pEnroll		// OUT파라미터 추출된 특징량이 저장될 공간
				// 시험도구에서는 최대 5000byte까지 허용함
	)
{
	// img로부터 영상을 읽어와 특징량을 추출한다.
	// 특징량을 아래 변수 Enroll에 담았다고 가정
	// 여기에서는 아무 값이나 대입하였음
	char Enroll[1000] = "5518bfe3fec4dbce392ddd4482812f7b";

	// 추출된 특징량을 OUT 파라미터로 지정된 pEnroll에
	// 아래와 같이 memcpy로 넣어준다.
	memcpy(pEnroll, Enroll, 1000);

	// 성공 시 true 리턴, 에러 시에는 false를 리턴한다.
	return true;
}

// 인식용 영상 이미지의 주소를 받아와 특징량을 추출하는 함수
// 인식용이라는 차이만 있을 뿐 KISA_Enroll함수와 처리가 동일
```

```
KISADLL bool KISA_Attempt(
        BYTE * img,                     // 인식용 영상 이미지 픽셀 주소
        int width,                      // 이미지 너비
        int height,                     // 이미지 높이
        BYTE * pAttempt                 // OUT파라미터 추출된 특징량이 저장될 공간
                                         // 시험도구에서는 최대 5000byte까지 허용함
        )
{
        // img로부터 영상을 읽어와 특징량을 추출한다.
        // 특징량을 아래 변수 Enroll에 담았다고 가정
        // 여기에서는 아무 값이나 대입하였음
        char Attempt[1000] = "5518bfe3fec4dbce392ddd4482812f7b";

        // 추출된 특징량을 OUT 파라미터로 지정된 pAttempt에
        // 아래와 같이 memcpy로 넣어준다.
        memcpy(pAttempt, Attempt, 1000);

        // 성공 시 true 리턴, 에러 시에는 false를 리턴한다.
        return true;
}

// 추출된 등록용 특징량과 인식용 특징량을 비교한 결과, 정합점수를 판정하는 함수
KISADLL float KISA_Compare(
        BYTE * pEnroll,         // KISA_Enroll 함수에서 넘겨줬던 특징량
        BYTE * pAttempt         // KISA_Attempt 함수에서 넘겨줬던 특징량
        )
{
        // pEnroll, pAttempt를 비교하여 정합 점수를 리턴한다.
        float fResult;          // 정합점수를 저장하기 위한 결과 변수. 이 값을 리턴한다.

        fResult = 0.5f;         // 정합점수가 0.5일 때 ( 0부터 1사이의 값 )

        return fResult;         // 도출된 정합점수 리턴
}
```

– 홍채인식 DLL 파일의 경로를 얻기 위한 구현 예제(DllMain.cpp)

```
HMODULE g_hDLL = NULL;
BOOL APIENTRY DllMain(HANDLE _hModule, DWORD _dwReason, LPVOID _lpRes)
{
	switch(_dwReason)
	{
	case DLL_PROCESS_ATTACH:
		g_hDLL = (HMODULE)_hModule;
		break;
	case DLL_PROCESS_DETACH:
		break;
	case DLL_THREAD_ATTACH:
		break;
	case DLL_THREAD_DETACH:
		break;
	}

	return TRUE;
}
// 이 함수처럼 구현하면 dll이 속해 있는 경로를 구할 수 있음.
void GetPath()
{
	char strPath [MAX_PATH] = {0};
	::GetModuleFileName(g_hDLL, strPath, MAX_PATH);
}
```

[별지 제1호 서식] 생체인식시스템 성능 시험・인증 신청서

<table>
<tr><td colspan="8">생체인식시스템 성능 시험・인증 신청서</td></tr>
<tr><td rowspan="7">신 청 인</td><td rowspan="2">상 호</td><td>한글</td><td colspan="5"></td></tr>
<tr><td>영어</td><td colspan="5"></td></tr>
<tr><td rowspan="2">주 소</td><td>한글</td><td colspan="5"></td></tr>
<tr><td>영어</td><td colspan="5"></td></tr>
<tr><td rowspan="2">대 표 자
성 명</td><td>한글</td><td></td><td>법인등록번호</td><td colspan="2"></td></tr>
<tr><td>영어</td><td></td><td>사업자등록번호</td><td colspan="2"></td></tr>
<tr><td rowspan="2">담 당 자</td><td colspan="2" rowspan="2"></td><td>전화</td><td colspan="2"></td></tr>
<tr><td>이메일</td><td colspan="2"></td></tr>
<tr><td>신청 유형</td><td colspan="7">☐ 사전시험 ☐ 본 시험 (1개만 선택 가능)</td></tr>
<tr><td rowspan="2">신청 제도
및 분야
(1개만 선택)</td><td>인증제도</td><td colspan="6">☐ 생체인식시스템 알고리즘 성능 시험·인증
☐ 생체인식시스템 표준적합성 시험·인증</td></tr>
<tr><td>분 야</td><td colspan="6">☐ 지문인식 ☐ 얼굴인식 ☐ 손등정맥인식 ☐ 지정맥인식
☐ 장정맥인식 ☐ 홍채인식</td></tr>
<tr><td rowspan="3">제품정보</td><td rowspan="2">명 칭</td><td>한글</td><td colspan="2"></td><td rowspan="2">버전</td><td>한글</td><td></td></tr>
<tr><td>영어</td><td colspan="2"></td><td>영어</td><td></td></tr>
<tr><td>정 보</td><td>탐지율</td><td colspan="5">%
(0~100% 가정시, 인증률)</td></tr>
<tr><td colspan="2">제출물</td><td colspan="6">☐ 생체인식시스템 성능 시험·인증
- 시험대상 제품(알고리즘) 1개, 센서 1개</td></tr>
<tr><td colspan="8">※ 한글 또는 영문으로만 작성
상기와 같이 시험을 신청하며, 기재사항에 허위사실이 없음을 확인합니다.
재난, 재해, 질병, 천재지변 및 경영환경 변화 등 불가피한 사유가 발생한 경우 시험이 중단 또는 지연될 수 있음을 확인하였습니다.

년 월 일

신 청 인 (서명 또는 인)

한국인터넷진흥원장 귀하</td></tr>
</table>

[별지 제2호 서식] 생체인식 방어력 성능 시험・인증 신청서

위조 생체인식 방어력 성능 시험・인증 신청서

신 청 인	상 호	한글				
		영어				
	주 소	한글				
		영어				
	대 표 자 성 명	한글		법인등록번호		
		영어		사업자등록번호		
	담 당 자			전화		
				이메일		
신청 유형	□ 사전시험 □ 본 시험 (1개만 선택 가능)					
신청 제도 및 분야	분 야	□ 지문인식 ※ 위조 생체인식 방어력 성능 시험·평가는 지문인식만 선택 가능				
제품정보	제품명	한글				
		영어				
	모델명	한글		버전		
		영어				
위조탐지 수준	제품옵션	탐지 강도	시험진행 탐지강도(0~10) :	탐지률	% (0~100% 가정시, 인증률)	
제출물		□ 위조 생체인식 방어력 성능 시험·평가 - 제품 메뉴얼 1부, 시험대상 제품 5개, 노트북 5대				

※ 한글 또는 영문으로만 작성

상기와 같이 시험을 신청하며, 기재사항에 허위사실이 없음을 확인합니다.
재난, 재해, 질병, 천재지변 및 경영환경 변화 등 불가피한 사유가 발생한 경우 시험이 중단 또는 지연될 수 있음을 확인하였습니다.

년 월 일

신 청 인 (서명 또는 인)

한국인터넷진흥원장 귀하

[별지 제3호 서식] 보안서약서

보 안 서 약 서

제품명	
기관명	
사업자등록번호(법인등록번호)	
대표자명	

※ 본 보안서약서는 대표이사의 서명을 원칙으로 하며, 대표이사로부터 위임 또는 승낙을 받은 경우 기관 명의의 공문(위임 내용 포함) 혹은 위임장을 통해 예외적으로 위임(대리)자의 작성이 가능함

※ 공동대표이사가 존재하는 경우, 전체 공동대표이사에 대한 서명 필요

본 신청기관은 한국인터넷진흥원에서 수행하는 생체인식시스템 시험·인증을 수행함에 있어 관련 진흥원 지침 및 국가 법령을 성실히 준수합니다.

또한, 생체인식시스템 시험 수행과정에서 취득한 사실과 제공받은 데이터를 성능 시험 수행 이외의 용도로 사용하거나 유출 또는 공개하지 아니할 것이며 위의 사항을 위반할 경우 성능시험 취소 및 관련 법규에 따라 어떠한 처벌 및 배상도 감수할 것을 서약합니다.

년 월 일

성 명 : (서 명)

[별지 제4호 서식] 인증연장 신청서

인증연장 신청서

1. 신청인 :

2. 인증 연장 대상 제품

– 인증 내용 :

– 인증 번호 :

생체인식시스템 시험 인증지침에 의하여 상기 제품의 인증 연장을 신청합니다.

년 월 일

신 청 인 (서명 또는 인)

한국인터넷진흥원장 귀하

생체인식시스템

| 시험·인증 안내서 |

발 행 2025년 11월

인 쇄 2025년 11월

발행처 한국인터넷진흥원

주소 : 서울특별시 송파구 중대로 135
IT벤처타워 한국인터넷진흥원

전화 : (02) 405-6633

본 안내서 내용의 무단 전재를 금하며, 가공·인용할 때에는 반드시 한국인터넷진흥원 『생체인식시스템 시험·인증 안내서』 라고 출처를 밝혀야 합니다.